

GEOSPATIAL DATA ANALYSIS USING MACHINE LEARNING

PROJECT REPORT

Submitted by

SAKSHI SRIVASTAVA

[EA2432251010614]

Under the Guidance of

Dr.G.Babu

(Assistant Professor, Directorate of Online Education)

In partial fulfillment for the award of the degree of

MASTER OF COMPUTER APPLICATIONS



**DIRECTORATE OF ONLINE EDUCATION
SRM INSTITUTE OF SCIENCE &
TECHNOLOGY**

KATTANKULATHUR- 603 203

JUNE 2026

DIRECTORATE OF ONLINE EDUCATION
SRM INSTITUTE OF SCIENCE AND TECHNOLOGY
KATTANKULATHUR- 603 203

BONAFIDE CERTIFICATE

This Project report titled **“Geospatial Data Analysis Using Machine Learning”** is the Bonafide work of **Sakshi Srivastava [EA2432251010614]** who completed the Project work along with the ISRO-IIRS mentor. Certified further, that to the best of my knowledge the work reported herein does not form any other Project report or dissertation based on which a degree or award was conferred on an earlier occasion on this or any other candidate

Signature of Student

PROJECT OFFER LETTER

This is to certify that **SAKSHI SRIVASTAVA**
[EA2432251010614] has undergone Project work during the period of 8 weeks
under the guidance of ISRO-IIRS Professors.

This work has been carried out in partial fulfillment of the
requirements for the award of the degree of **MASTER OF COMPUTER APPLICATIONS**.



अंतरिक्ष विभाग / DEPARTMENT OF SPACE
भारत सरकार / GOVERNMENT OF INDIA
भारतीय अंतरिक्ष अनुसंधान संगठन / INDIAN SPACE RESEARCH ORGANISATION
भारतीय सुदूर संवेदन संस्थान, देहरादून / INDIAN INSTITUTE OF REMOTE SENSING, DEHRADUN



नामांकन सं. / Enrolment No. : 20261762931506



CERTIFICATE OF MERIT ONLINE COURSE

यह प्रमाणपत्र

MS. SAKSHI SRIVASTAVA

को " पाथचन और मशीन लर्निंग का उपयोग करके भू-डेटा प्रसंस्करण "

में ऑनलाइन पाठ्यक्रम पूर्ण करने पर प्रदान किया जाता है।

इस ऑनलाइन पाठ्यक्रम का आयोजन 2026-02-09 से 2026-02-26 (कुल पाठ्यक्रम की अवधि = 9 घंटे) के दौरान किया गया।

(संयन्त्रित आई.आई.आर.एस नोडल केंद्र- एसआरएम इंस्टीट्यूट ऑफ साइन्स एंड टेक्नोलॉजी (स्कूल ऑफ कंप्यूटिंग))

THIS CERTIFICATE IS

AWARDED TO

MS. SAKSHI SRIVASTAVA

ON COMPLETION OF THE ONLINE COURSE ON

"Geodata Processing using Python and Machine Learning"

THIS ONLINE COURSE WAS CONDUCTED DURING **09 February 2026 to 26 February 2026** (Total course duration = 9 hrs).

(Concerned IIRS Nodal Centre- SRM INSTITUTE OF SCIENCE & TECHNOLOGY(SCHOOL OF COMPUTING))

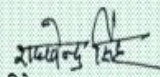
Online Exam Grade: **A+**

Attendance: **100 %**

Date: **08-05-2026**

Place: Dehradun

समन्वयक, विश्वविद्यालय/संस्थान
Coordinator, University/ Institution


निदेशक/ Director
आई.आई.आर.एस, देहरादून/ IIRS, Dehradun

UID- e072687534fee4baa2b79761fee42155. This Certificate can be validated using URL- <https://certificate.iirs.gov.in>

* >80% -{A+}, 70-80% -{A}, 60-70% -{B+}, 40-60% -{B}

TABLE OF CONTENTS

Sr. No.	Item Description	Page No.
1.	Abstract	6-8
2.	Introduction & Objective of Project	9-10
3	Problem Definition	11-18
4	Hardware Requirements	19-21
5	Software Requirements	12-14
6	Process Analysis and Solution	15-33
7	Implementations methodology	34-46
8	List of reports	40-48
9	Conclusion	48
10	References	49
11.	Appendices	50-57
	• Sample Source Code • Student Declarations Report	

ABSTRACT

The project titled **“Geospatial Data Analysis Using Machine Learning for Environmental Risk Prediction”** presents the development of an intelligent, cloud-enabled Geo-AI analytics framework designed to perform environmental monitoring, vegetation assessment, and spatial risk prediction using advanced Machine Learning and Geographic Information System (GIS) technologies. The increasing availability of satellite imagery, environmental datasets, and cloud computing resources has created new opportunities for building scalable and automated geospatial analytics systems capable of supporting real-time environmental decision-making. This project aims to address the limitations of traditional environmental analysis methods, which are often manual, time-consuming, computationally expensive, and less effective in handling large-scale spatial datasets.

The proposed system integrates geospatial data processing, Machine Learning algorithms, interactive visualization, and secure web technologies into a unified analytics platform. The framework utilizes multiple open-source geospatial datasets, including multispectral satellite imagery from Sentinel-2 and Landsat-8, vector-based spatial information from OpenStreetMap, Digital Elevation Models (DEM), and environmental datasets inspired by ISRO-IIRS geospatial applications. These datasets provide rich spatial and temporal information required for environmental analysis and predictive modeling.

The workflow begins with geospatial data acquisition and preprocessing. Since raw geospatial datasets often contain inconsistencies such as cloud cover, sensor noise, missing values, varying coordinate systems, and spatial distortions, the system performs extensive preprocessing operations including georeferencing, coordinate system harmonization, raster normalization, cloud masking, noise filtering, spatial clipping, and missing data handling. Feature engineering techniques are then applied to derive meaningful environmental parameters from raw geospatial data.

Several important spatial features are extracted and generated, including:

- **Normalized Difference Vegetation Index (NDVI)** for vegetation health monitoring,
- **Land Surface Temperature (LST)** for thermal analysis,
- **Elevation and slope** derived from Digital Elevation Models,
- **Rainfall and climatic parameters** for environmental trend analysis,
- **Land use and land cover features** for spatial classification,
- **Spatial proximity and terrain attributes** for environmental risk estimation.

The processed data is then utilized for supervised Machine Learning tasks such as land cover classification, environmental risk prediction, vegetation stress detection, and environmental change analysis. The project employs multiple Machine Learning algorithms including:

- **Random Forest**
- **Gradient Boosting**
- **Support Vector Machines (SVM)**
- **Decision Trees**
- **Convolutional Neural Networks (CNNs)** for spatial image-based classification.

These algorithms are trained and evaluated using historical environmental datasets to identify hidden spatial patterns and generate predictive insights. The Machine Learning models are capable of detecting environmental changes such as deforestation, urban expansion, vegetation degradation, and high-risk ecological zones.

To ensure model reliability and prediction accuracy, the system performs detailed performance evaluation using metrics such as:

- Accuracy,
- Precision,
- Recall,
- F1-Score,
- Root Mean Square Error (RMSE),

- Mean Absolute Error (MAE),
- Confusion Matrix analysis.

The project also incorporates interactive geospatial visualization and analytical reporting features. Dynamic maps and spatial prediction layers are generated using Python GIS libraries and web-mapping technologies such as Leaflet.js. Analytical graphs and statistical representations are visualized using Chart.js, enabling users to better interpret environmental patterns and prediction outcomes. Users can interactively explore spatial risk zones, vegetation conditions, and classification results through a user-friendly dashboard interface.

For deployment and accessibility, the system is implemented as a cloud-based web application using Flask and Streamlit frameworks with REST-based architecture. The platform allows users to upload geospatial datasets, perform environmental analysis, and obtain real-time prediction results through an intuitive web dashboard. JWT-based authentication and secure user management mechanisms are integrated to ensure data security and controlled access.

The developed Geo-AI analytics platform demonstrates how Artificial Intelligence, Remote Sensing, and GIS technologies can be effectively integrated to build scalable environmental monitoring systems. The proposed framework provides significant applications in:

- Precision agriculture,
- Disaster management,
- Climate change monitoring,
- Urban planning,
- Forest and vegetation management,
- Sustainable environmental development.

By combining Machine Learning with geospatial analytics and cloud computing technologies, the project contributes toward the advancement of intelligent environmental decision support systems capable of handling large-scale spatial datasets efficiently.

The proposed solution offers a scalable, automated, and data-driven approach for

environmental risk assessment and geospatial prediction, supporting future smart environmental management initiatives and sustainable development goals.

INTRODUCTION

1. INTRODUCTION

Geospatial data analysis has become one of the most important research domains in modern computing systems due to the increasing demand for environmental monitoring, disaster management, and climate analysis. Traditional systems used for geospatial analysis generally depend on manual interpretation and static mapping systems, which often fail to provide intelligent prediction capabilities.

Artificial Intelligence and Machine Learning have introduced new possibilities for intelligent geospatial analysis by enabling systems to automatically identify patterns, trends, and anomalies from spatial datasets. Machine Learning algorithms can analyze large environmental datasets and generate predictive insights that support better decision-making.

The proposed Geo-AI Analytics Dashboard is designed to integrate Machine Learning techniques with geospatial visualization systems to predict environmental risk zones. The dashboard allows users to upload environmental datasets and obtain prediction results through charts and geospatial maps.

The system aims to provide a modern analytical platform inspired by advanced space research dashboards and geospatial intelligence systems. The project demonstrates the practical application of Artificial Intelligence in environmental analytics and geospatial computing.

1.1 Objectives

Geospatial data refers to information that is associated with a specific geographic location. It plays a vital role in multiple domains such as agriculture, environmental monitoring, disaster management, urban planning, and resource management. With the rapid growth of satellite imagery and Geographic Information Systems (GIS), the volume of geospatial data has increased significantly.

However, analyzing such data manually is difficult due to its complexity and large size. Traditional methods are often time-consuming and require domain expertise. This creates a need for intelligent systems that can process and analyze geospatial data efficiently. Machine Learning (ML) provides a solution by enabling systems to learn patterns from data and make predictions. By applying ML techniques to geospatial data, it becomes possible to automate analysis and generate meaningful insights.

1.2 Motivation

The motivation behind this project includes:

- Increasing availability of geospatial data
- Need for automated analysis tools
- Limitations of manual data processing
- Growing demand for data-driven decision-making

This project aims to bridge the gap between raw geospatial data and intelligent analysis.

1.3 Objectives

The main objectives of the project are:

- To develop a system for analyzing geospatial data
 - To apply machine learning techniques for prediction
 - To automate data preprocessing and analysis
 - To provide a user-friendly interface for users
 - To generate accurate and meaningful results
-

1.4 Scope of the Project

The project focuses on:

- Structured geospatial datasets (CSV format)
- Machine learning-based prediction
- Basic visualization of results

It does not include:

- Real-time satellite data processing
 - Advanced GIS software integration
-

1.5 Applications

- Agriculture monitoring
- Environmental analysis
- Land classification
- Climate studies

PROBLEM DESCRIPTION

2. PROBLEM DESCRIPTION

Environmental monitoring systems continuously generate massive volumes of geospatial and environmental data through satellite imagery, remote sensing platforms, climate sensors, and Geographic Information Systems (GIS). These datasets contain valuable information related to vegetation health, land use patterns, climate conditions, rainfall distribution, temperature variations, and environmental changes. However, traditional methods used for analyzing such datasets are largely manual, computationally intensive, and time-consuming, making them inefficient for real-time environmental monitoring and decision-making.

Conventional environmental analysis systems primarily focus on static data visualization and basic spatial analysis, with limited capability to perform intelligent prediction or automated risk assessment. As environmental datasets continue to grow in size and complexity, existing systems struggle to process large-scale geospatial information efficiently and provide accurate predictive insights in real time.

The major limitations of existing environmental monitoring and geospatial analysis systems include:

- **Lack of intelligent prediction mechanisms:**
Most traditional GIS platforms are designed for visualization and mapping purposes only, without incorporating Machine Learning or Artificial Intelligence techniques for predictive environmental analysis.
- **Limited integration between Machine Learning and GIS technologies:**

Existing systems often fail to effectively combine geospatial analytics with Machine Learning models, reducing the ability to identify hidden spatial patterns and predict environmental risks accurately.

- **Inability to provide interactive geospatial visualization:**
Many systems lack dynamic dashboards and interactive mapping features required for effective interpretation of prediction results and spatial trends.
- **Poor scalability for large geospatial datasets:**
Processing satellite imagery, raster datasets, and environmental parameters requires high computational capability, which many traditional systems cannot efficiently handle.
- **Limited real-time analytical and decision-support capabilities:**
Existing platforms are often unable to generate real-time predictions and environmental risk assessments necessary for disaster management, agricultural planning, and climate monitoring.
- **Complexity in handling heterogeneous spatial datasets:**
Environmental datasets originate from multiple sources with different formats, coordinate systems, and resolutions, making integration and preprocessing difficult in conventional systems.
- **Insufficient support for automated environmental change detection:**
Traditional methods require significant manual intervention for identifying changes such as deforestation, urban expansion, vegetation degradation, and climate-related risks.

To overcome these limitations, there is a need for an intelligent and scalable Geo-AI analytics platform capable of integrating geospatial data processing, Machine Learning algorithms, cloud computing, and interactive visualization technologies into a unified system. The proposed system aims to efficiently process environmental datasets, generate predictive insights, identify environmental risk zones, and provide interactive geospatial visualizations for effective environmental monitoring and decision support.

The developed solution will support applications in agriculture, disaster management, climate risk assessment, urban planning, and sustainable environmental development by enabling accurate, automated, and real-time geospatial analytics.

2.2 Challenges

Developing an intelligent geospatial analytics platform involves several technical and computational challenges.

Some of the major challenges faced during the project are described below:

1. Handling Large Geospatial Datasets

Satellite imagery and environmental datasets are usually very large in size and contain high-dimensional spatial information. Efficient storage, processing, and analysis of such datasets require optimized computational techniques.

2. Data Preprocessing Complexity

Raw geospatial data often contains missing values, cloud cover, noise, inconsistent coordinate systems, and varying spatial resolutions. Preprocessing and cleaning these datasets is a complex and time-consuming task.

3. Feature Extraction and Engineering

Extracting meaningful environmental features such as NDVI, temperature, elevation, and rainfall from raw satellite data requires specialized geospatial processing methods.

4. Machine Learning Model Selection

Selecting suitable Machine Learning algorithms for environmental prediction and land classification is challenging because different models perform differently depending on the dataset and spatial characteristics.

5. Accuracy Improvement

Environmental prediction systems require high accuracy and reliability. Improving model performance while reducing overfitting and computational complexity is an important challenge.

6. Integration of GIS and Machine Learning

Combining spatial analysis techniques with Machine Learning models into a unified framework requires efficient integration between geospatial processing tools and AI technologies.

7. Real-Time Prediction and Visualization

Generating real-time predictions and displaying them through interactive geospatial dashboards requires efficient backend processing and visualization mechanisms.

8. Scalability and Deployment

The system should be scalable enough to support large environmental datasets and cloud deployment for multiple users and real-time analytics.

3.3 Proposed Solution

To overcome the limitations of traditional environmental monitoring systems, the proposed project develops an intelligent and automated Geo-AI analytics platform using Machine Learning and GIS technologies.

The proposed solution is designed as a cloud-enabled web application capable of processing geospatial datasets, applying Machine Learning models, generating predictions, and visualizing environmental insights interactively.

The system performs the following operations:

- Accepts environmental and geospatial datasets as input
- Performs automated data preprocessing and cleaning
- Extracts important environmental features
- Applies Machine Learning algorithms for prediction and classification
- Generates environmental risk predictions and analytical reports
- Displays interactive visualizations including maps, charts, and statistical outputs
- Supports secure and scalable web-based access

The proposed solution integrates:

- Python-based geospatial processing
- Machine Learning algorithms
- GIS visualization technologies
- Interactive dashboards
- Cloud deployment architecture

The system provides an efficient and intelligent framework for applications such as:

- Environmental monitoring
- Vegetation analysis
- Disaster prediction
- Climate risk assessment
- Urban expansion monitoring
- Agricultural planning

By integrating Artificial Intelligence with geospatial analytics, the proposed solution improves prediction accuracy, automation, scalability, and decision-support capabilities.

2.3 MODULES DESCRIPTION

Data Input Module

The Data Input Module is responsible for collecting and loading environmental datasets into the system. This module allows users to upload geospatial and environmental data files for analysis and prediction.

Functions of the Module:

- Accepts CSV datasets containing environmental parameters
- Supports structured geospatial data input
- Validates dataset format and structure
- Reads uploaded files into the system
- Verifies required attributes such as latitude, longitude, rainfall, temperature, and vegetation index
- Prevents invalid or corrupted data uploads

Technologies Used:

- Python
- Pandas
- Flask/Streamlit file upload components

Data Preprocessing Module

The Data Preprocessing Module prepares raw geospatial and environmental data for Machine Learning analysis. Since environmental datasets often contain noise, inconsistencies, and missing values, preprocessing is essential for improving prediction accuracy.

Functions of the Module:

- Handles missing and null values
- Removes duplicate and inconsistent records
- Performs data normalization and scaling

- Encodes categorical variables if required
- Performs feature extraction and feature selection
- Generates environmental indices such as NDVI
- Splits datasets into training and testing data

Benefits:

- Improves data quality
- Reduces computational complexity
- Enhances Machine Learning performance

Technologies Used:

- Pandas
 - NumPy
 - Scikit-learn preprocessing libraries
-

Machine Learning Module

The Machine Learning Module is the core analytical component of the system. It applies Machine Learning algorithms to identify environmental patterns and generate predictive insights.

Functions of the Module:

- Performs model training using historical environmental data
- Implements classification and prediction algorithms
- Detects environmental risk zones
- Performs land cover classification
- Evaluates model performance using standard metrics

Algorithms Used:

- Random Forest
- Decision Tree
- Gradient Boosting
- Support Vector Machine (SVM)

- Convolutional Neural Network (CNN)

Evaluation Metrics:

- Accuracy
- Precision
- Recall
- F1-Score
- RMSE
- Confusion Matrix

Technologies Used:

- Scikit-learn
 - TensorFlow/Keras
 - Python
-

Prediction Module

The Prediction Module generates environmental predictions using trained Machine Learning models.

Functions of the Module:

- Accepts new environmental input data
- Applies trained models for prediction
- Predicts environmental risk levels
- Identifies vegetation stress zones
- Generates prediction outputs in real time

Outputs Generated:

- Risk prediction
- Classification results
- Environmental status reports

Benefits:

- Supports real-time environmental monitoring₁

- Assists decision-making and planning
-

Visualization Module

The Visualization Module provides graphical and geospatial representation of analytical results and prediction outputs.

Functions of the Module:

- Displays interactive geospatial maps
- Generates charts and graphs
- Visualizes prediction distributions
- Displays statistical reports and analytical summaries
- Supports spatial risk mapping

Visualization Technologies:

- Leaflet.js
- Chart.js
- Matplotlib
- Plotly

Benefits:

- Improves data interpretation
 - Enhances user understanding
 - Supports interactive environmental analysis
-

User Interface Module

The User Interface Module provides a simple and user-friendly environment for interacting with the system.

Functions of the Module:

- Allows dataset upload
- Displays prediction results
- Shows interactive maps and charts

- Enables user interaction with analytical dashboards
- Provides secure login and authentication

Features:

- Responsive web dashboard
- Interactive controls
- Real-time result display
- User-friendly design

Technologies Used:

- HTML
- CSS
- JavaScript
- Flask
- Streamlit

The User Interface Module acts as the communication layer between users and the Geo-AI analytics system, ensuring smooth interaction and accessibility.

Tools/Platform Hardware and Software Requirement specifications.

3 Tools/Platform- Hardware and Software Requirement specifications

3.1 Hardware Requirements:-

Component	Specification
Processor	Intel i3 or above
RAM	Minimum 4GB
Storage	20GB Free Space
Internet	Required

3.2 Software Requirements:-

Component	Specification
Processor	Intel i3 or above
RAM	Minimum 4GB
Storage	20GB Free Space
Internet	Required

SOFTWARE REQUIREMENT SPECIFICATION (SRS)

Functional Requirements

The system must provide the following functionalities:

1. User Registration and Login
2. Secure Authentication using JWT
3. CSV Dataset Upload
4. Environmental Risk Prediction
5. Data Visualization through Graphs
6. Geospatial Map Visualization
7. User Prediction History
8. Admin Dashboard
9. Logout and Session Management

Non-Functional Requirements

1. Security
2. Scalability
3. Reliability
4. Performance
5. User Friendliness
6. Cross-Platform Compatibility

SRS (SOFTWARE REQUIREMENTS SPECIFICATION)

1. FUNCTIONAL REQUIREMENTS

ID	Requirement	Description
FR1	User Registration & Login	System shall allow users to register and login securely.
FR2	User Management (Admin)	Admin shall be able to add, update, delete and view users.
FR3	Dataset Upload	Users shall be able to upload CSV datasets.
FR4	Data Preprocessing	System shall preprocess the uploaded data (cleaning, handling missing values).
FR5	Model Training	System shall train machine learning models on the dataset.
FR6	Prediction	System shall generate predictions on the dataset.
FR7	Visualization	System shall display prediction results using charts and maps.
FR8	Reports	System shall generate and export reports (PDF/Excel).
FR9	History	System shall maintain history of datasets and predictions.
FR10	Logout	System shall allow users to logout securely.

2. NON-FUNCTIONAL REQUIREMENTS

ID	Requirement	Description
NFR1	Performance	System shall provide results within acceptable response time.
NFR2	Reliability	System shall be reliable and available 24/7.
NFR3	Usability	System shall be easy to use with a user-friendly interface.
NFR4	Security	System shall ensure authentication and data security.
NFR5	Scalability	System shall handle large datasets and multiple users.
NFR6	Compatibility	System shall be compatible across major browsers.
NFR7	Maintainability	System shall be modular and easy to maintain.
NFR8	Portability	System shall be platform independent (Web-based).

PROBLEM DEFINITION

4 Function Requirements– DESIGN

Design is the first step in the development phase for any techniques and principles for the purpose of defining a device, a process or system in sufficient detail to permit its physical realization.

Once the software requirements have been analyzed and specified the software design involves three technical activities - design, coding, implementation and testing that are required to build and verify the software.

The design activities are of main importance in this phase, because in this activity, decisions ultimately affecting the success of the software implementation and its ease of maintenance are made. These decisions have the final bearing upon reliability and maintainability of the system. Design is the only way to accurately translate the customer's requirements into finished software or a system.

Design is the place where quality is fostered in development. Software design is a process through which requirements are translated into a representation of software. Software design is conducted in two steps. Preliminary design is concerned with the transformation of requirements into data.

a. UML Diagrams:

UML stands for Unified Modeling Language. UML is a language for specifying, visualizing and documenting the system. This is the step while developing any product after analysis. The goal from this is to produce a model of the entities involved in the project which later need to be built. The representation of the entities that are to be used in the product being developed need to be designed.

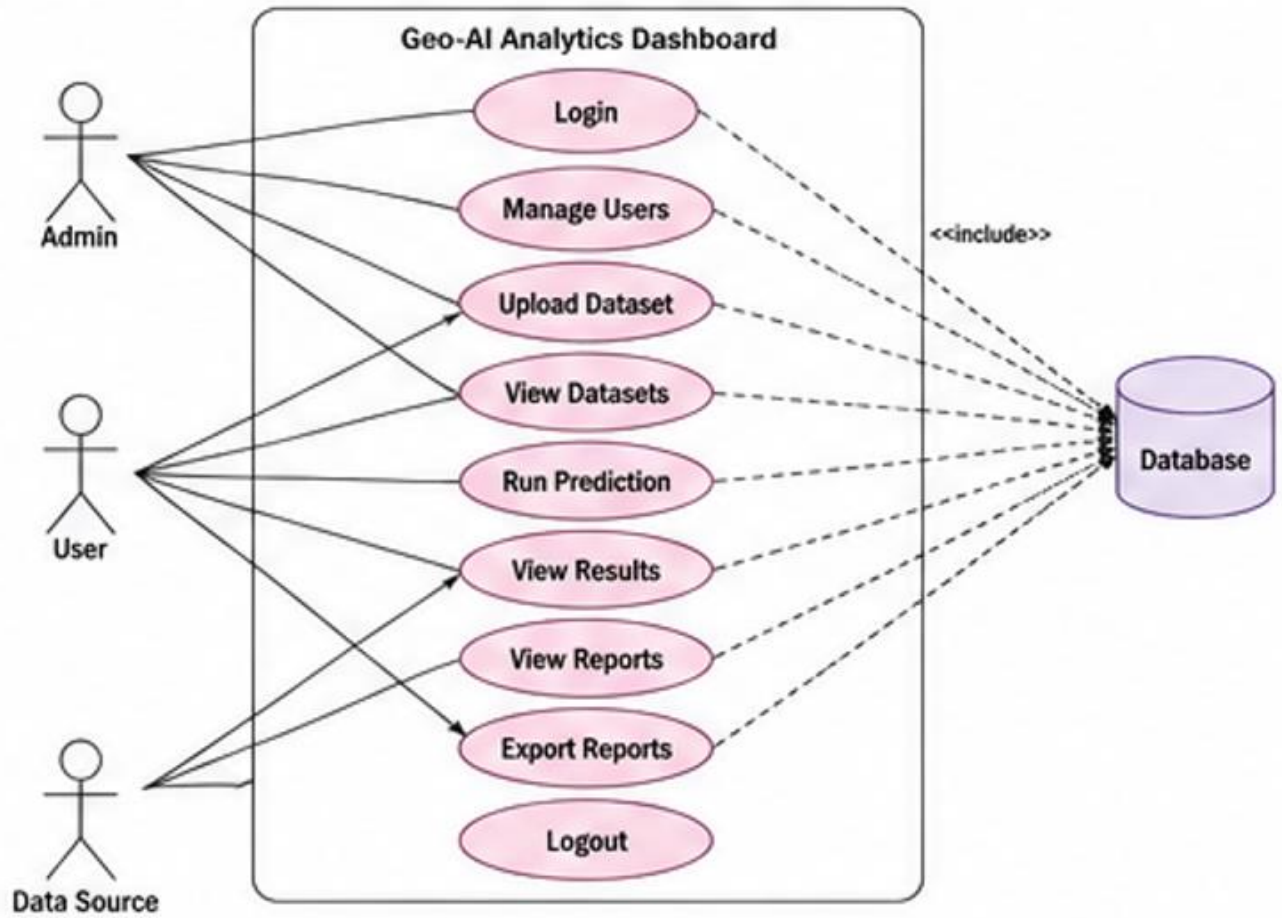
There are various kinds of methods in software design:

- Use case Diagram
- Sequence Diagram
- Collaboration Diagram

i. Usecase Diagrams:

Use case diagrams model behavior within a system and helps the developers understand of what the user require. The stick man represents what's called an actor. Use case diagram can be useful for getting an overall view of the system and clarifying who can do and more importantly what they can't do.

USE CASE DIAGRAM



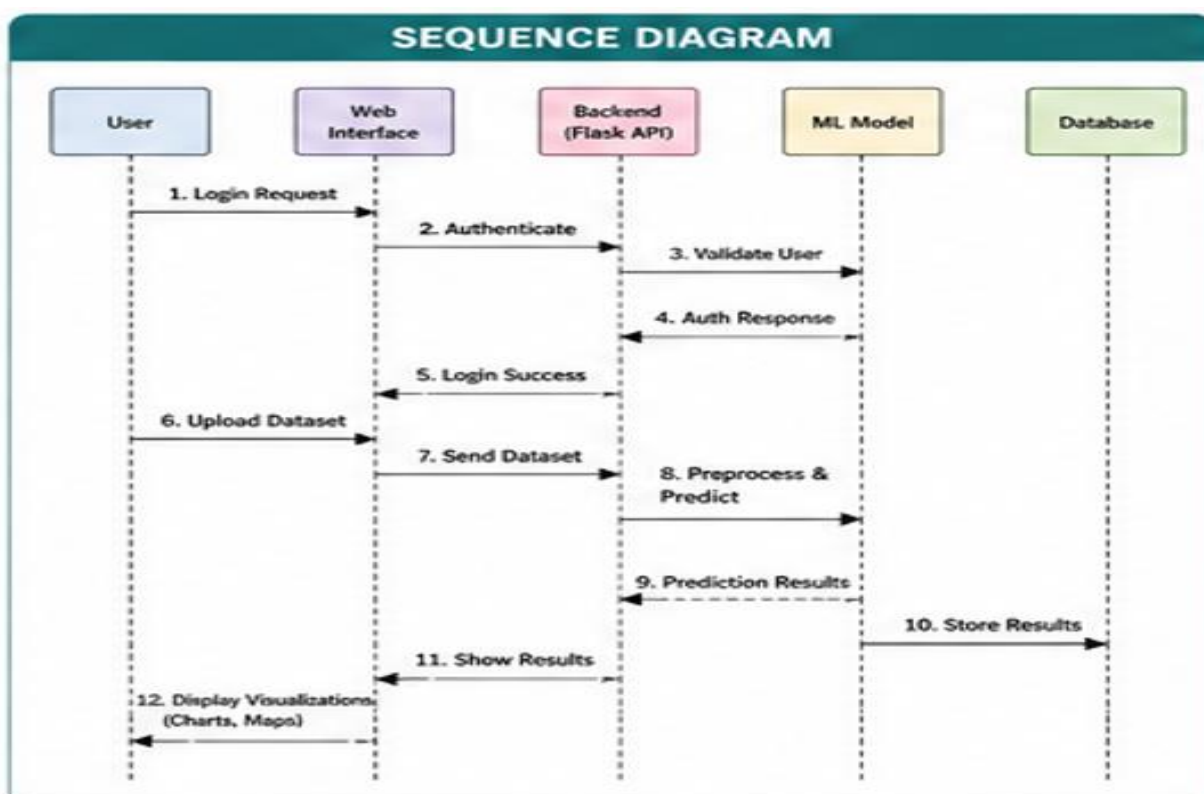
Use case diagram consists of use cases and actors and shows the interaction between the use case and actors.

- The purpose is to show the interactions between the use case and actor.
- To represent the system requirements from user's perspective.
- An actor could be the end-user of the system or an external system

ii. Sequence Diagram:

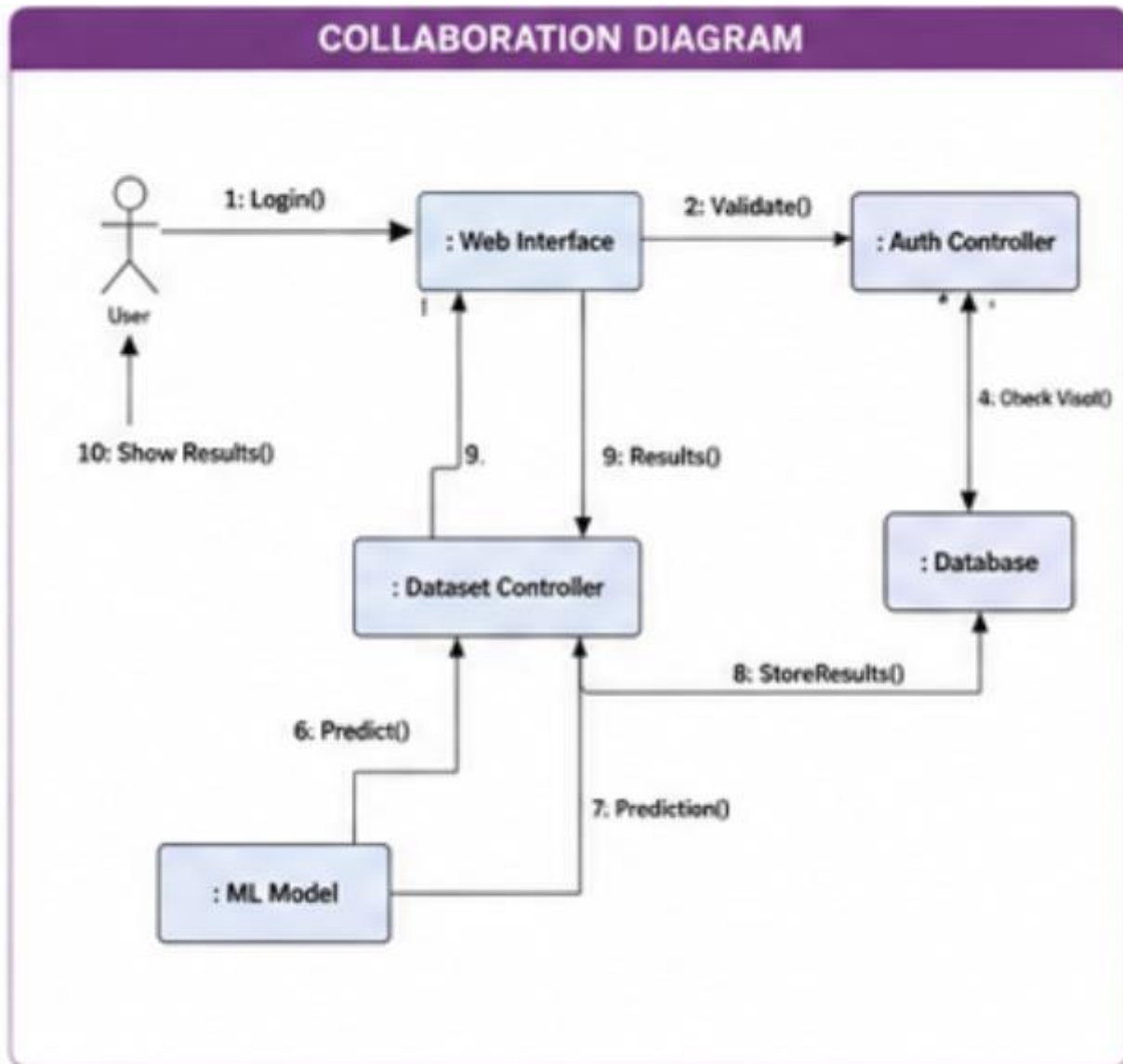
Sequence diagram and collaboration diagram are called INTERACTION DIAGRAMS. An interaction diagram shows an interaction, consisting of set of objects and their relationship including the messages that may be dispatched among them.

A sequence diagram is an introduction that empathizes the time ordering of messages. Graphically a sequence diagram is a table that shows objects arranged along the X-axis and messages ordered in increasing time along the Y- axis.



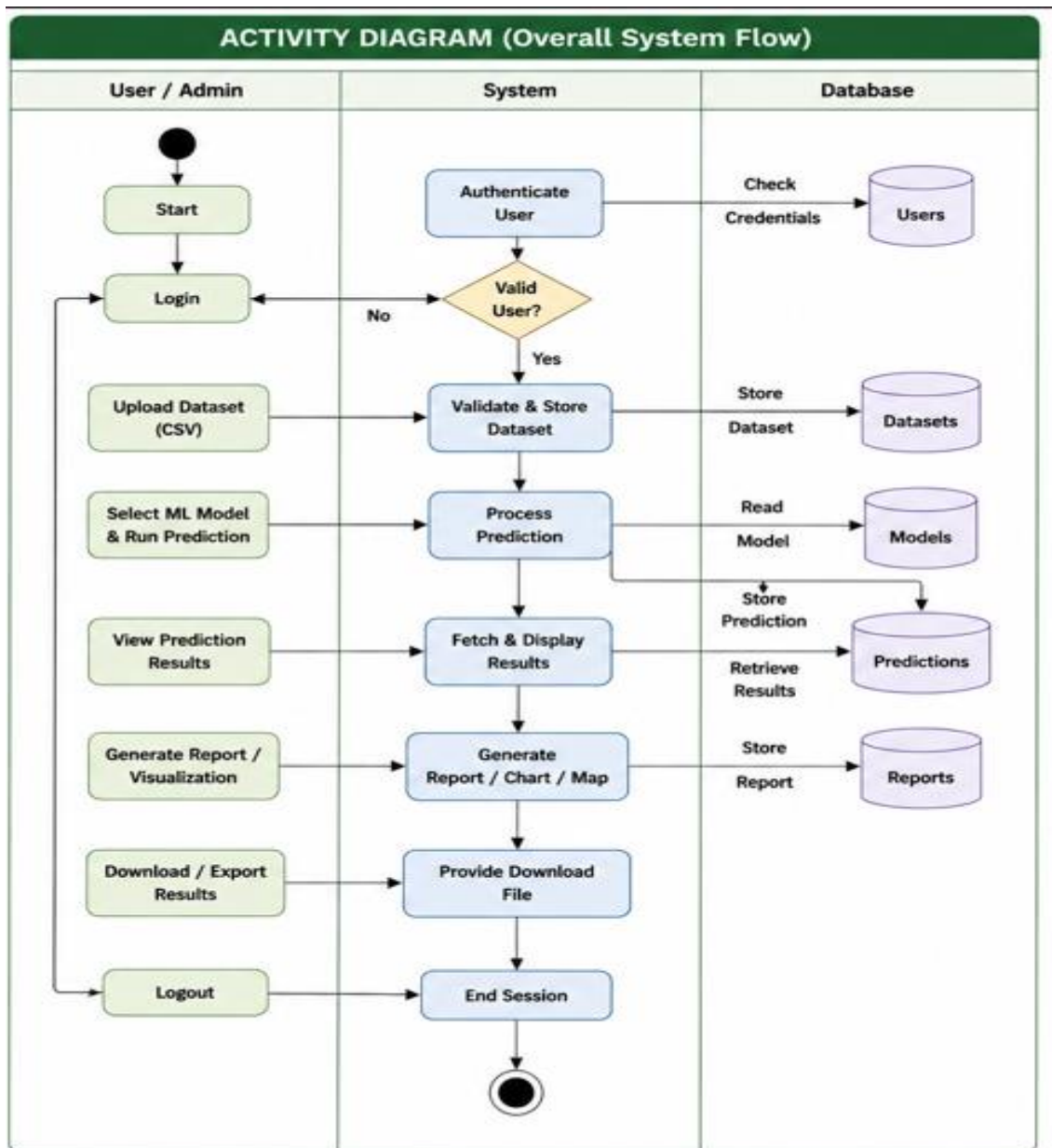
iii. Collaboration Diagram:

A **collaboration diagram** is a type of visual presentation that shows how various software objects interact with each other within an overall IT architecture and how users can benefit from this **collaboration**. A **collaboration diagram** often comes in the form of a visual chart that resembles a flow chart.

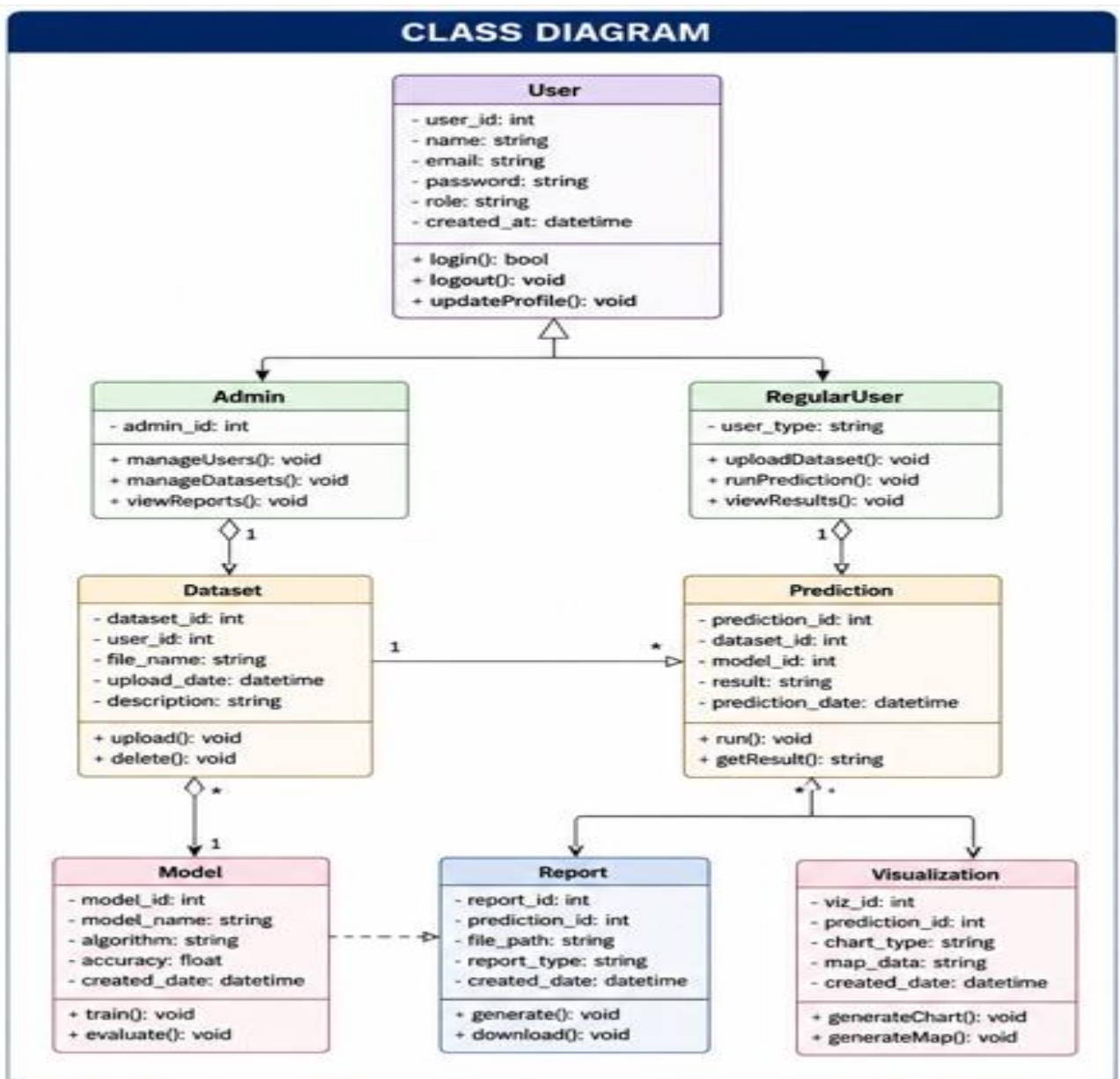


iv. Activity Diagram

Activity diagram is another important diagram in UML to describe dynamic aspects of the system. Activity diagram is basically a flow chart to represent the flow from one activity to another activity. The activity can be described as an operation of the system. So the control flow is drawn from one operation to another.

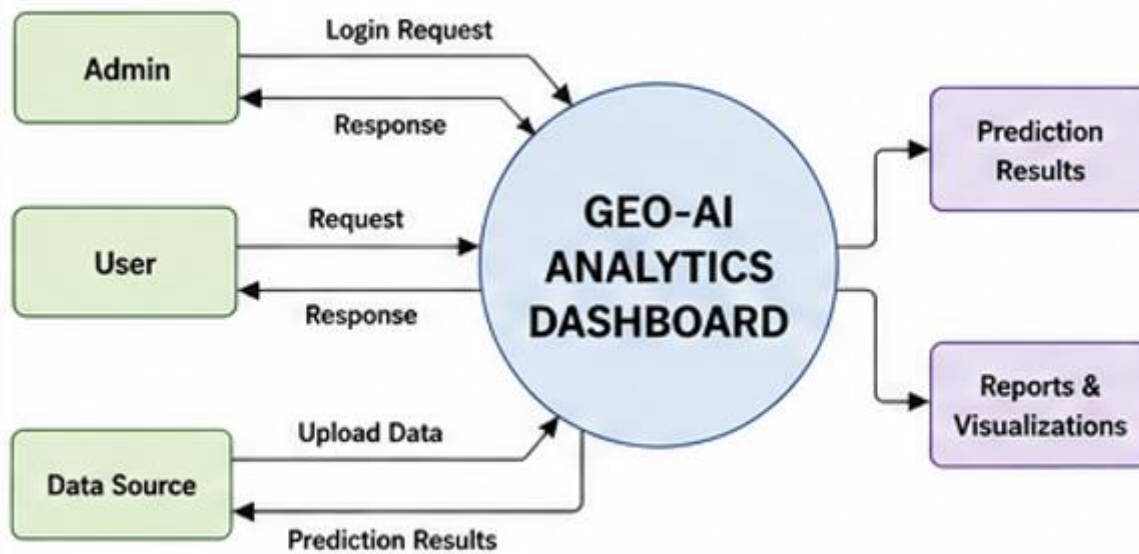


v. Class Diagram

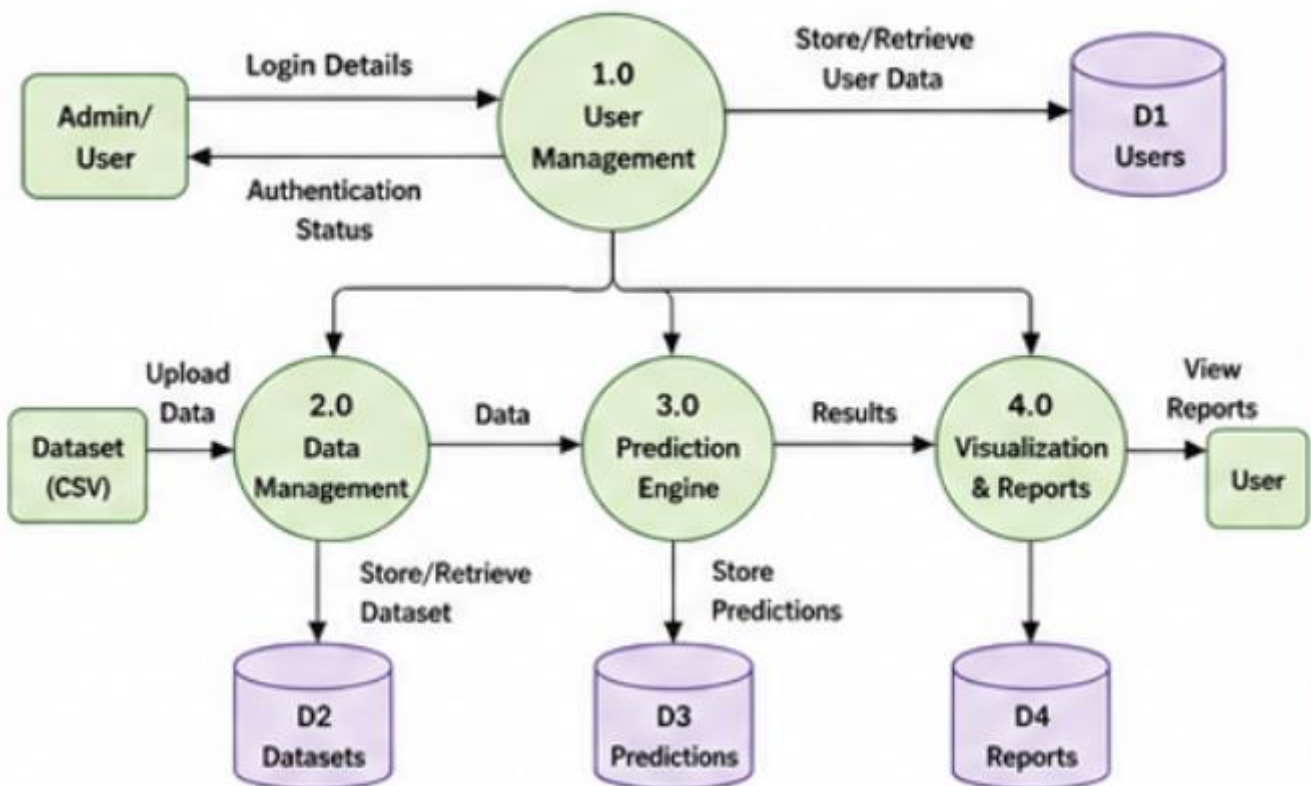


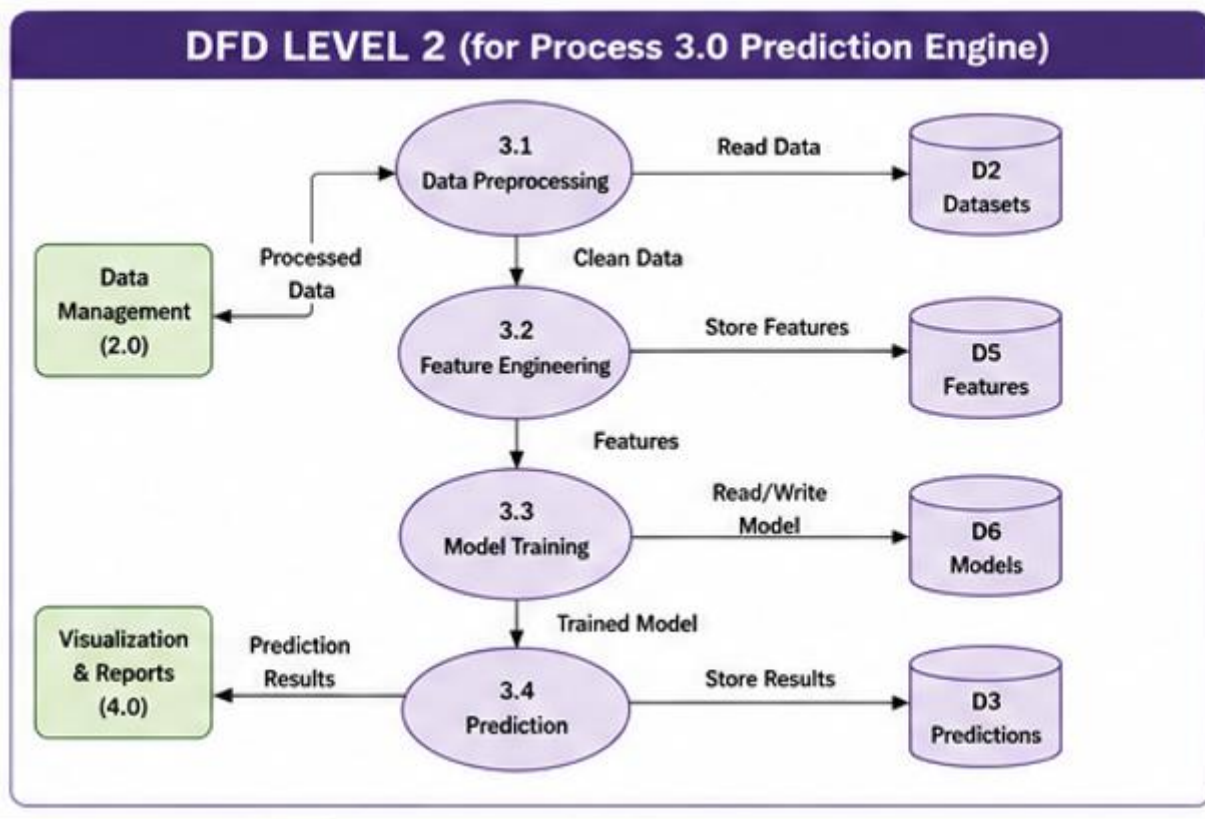
vi. DFD Diagram

DFD LEVEL 0 (CONTEXT DIAGRAM)



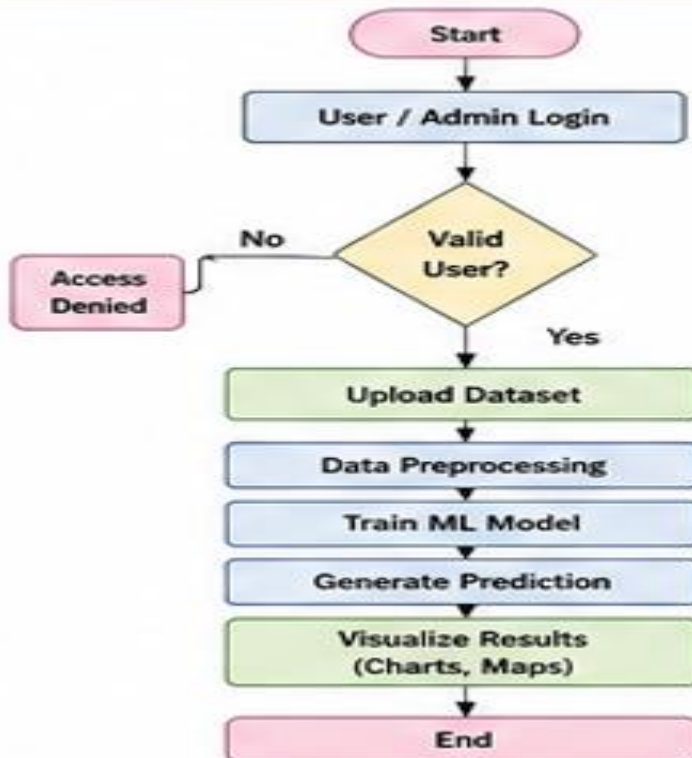
DFD LEVEL 1



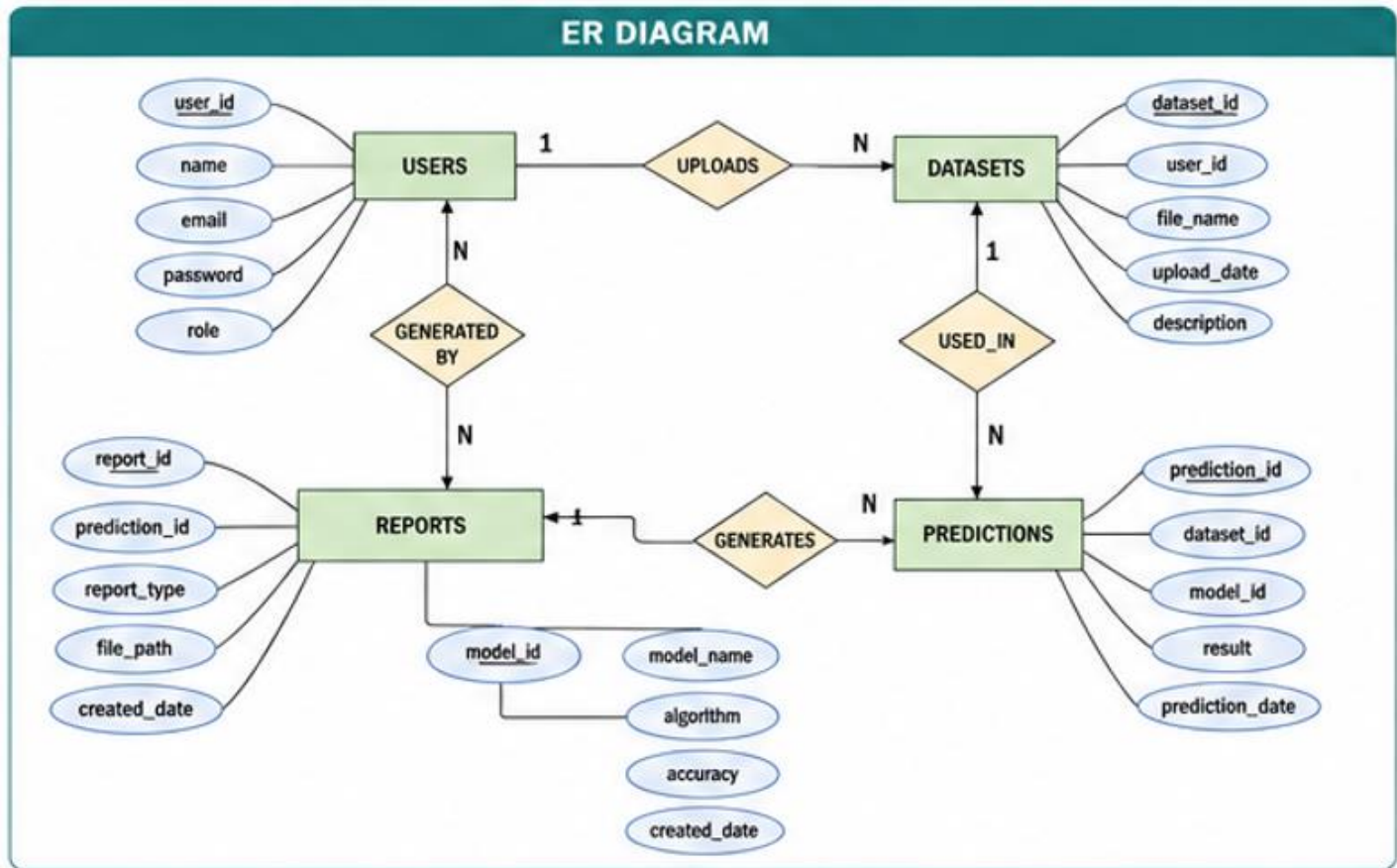


i. WORKFLOW DIAGRAM

FLOWCHART (OVERALL SYSTEM FLOW)



ER DIAGRAM



SCOPE OF ANALYSIS AND SOLUTION

4 SCOPE OF ANALYSIS AND SOLUTION

Existing Solution:

Existing environmental monitoring systems are primarily focused on GIS-based visualization and manual data interpretation. While these systems can display spatial data, they do not provide predictive analytics or interactive analysis tools.

Drawbacks of the Existing System:

- **Manual environmental analysis:** Data processing and interpretation are time-consuming and prone to human error.
- **Lack of predictive analytics:** Systems cannot forecast environmental risk zones, vegetation stress, or

climate-related impacts.

- **Limited visualization capabilities:** Static maps and reports fail to provide dynamic insights.
- **Poor user interaction:** Minimal interactivity reduces usability and decision-making efficiency.
- **Lack of secure authentication:** Existing systems often lack proper user management and data protection mechanisms.

The inefficiencies of existing systems create a need for a solution that integrates intelligent analytics, interactive visualization, and secure cloud deployment.

Proposed Solution:

The proposed system is an AI-driven geospatial analytics platform designed to overcome the limitations of traditional environmental monitoring systems. It integrates Machine Learning, GIS, interactive dashboards, and secure web technologies to provide intelligent environmental predictions and visualizations.

Features of the Proposed System:

- **Machine Learning-based prediction:** Supports land cover classification, vegetation risk prediction, and environmental change detection.
- **Interactive map visualization:** Provides dynamic geospatial maps using Leaflet.js for real-time spatial analysis.
- **Dashboard-based analytics:** Offers a centralized interface to view prediction results, charts, and historical data.
- **JWT authentication:** Ensures secure user login, access control, and data protection.
- **Prediction history management:** Allows users to track past analyses and model outputs for future reference.
- **User-friendly interface:** Intuitive dashboard design for easy dataset upload, analysis, and interpretation.

By combining AI and GIS technologies, the system enhances automation, prediction accuracy, interactivity, and security, making it suitable for applications in environmental monitoring, disaster management, agriculture, and urban planning.

Advantages

The proposed AI-driven geospatial analytics platform provides several significant advantages over traditional environmental monitoring systems:

1. Intelligent Prediction Capabilities

- Utilizes Machine Learning algorithms (Random Forest, Gradient Boosting, CNN, etc.) to predict environmental risks, vegetation health, land cover changes, and climate-related patterns.
 - Provides data-driven insights, reducing reliance on manual interpretation.
-

2. Interactive Geospatial Visualization

- Generates dynamic maps and charts using Leaflet.js and Chart.js.
 - Allows users to interact with spatial data, zoom into specific areas, and explore environmental patterns in real time.
 - Supports visualization of predictive layers, historical trends, and risk zones.
-

3. Automated Data Processing

- Handles large and complex geospatial datasets efficiently.
 - Preprocessing tasks such as missing value handling, normalization, and feature extraction are automated, reducing manual workload.
 - Ensures consistency and accuracy in analysis.
-

4. Scalable and Cloud-Enabled

- The system is designed for cloud deployment, enabling remote access and collaboration.
 - Can handle large-scale environmental datasets without performance degradation.
 - Supports multiple concurrent users through secure web interfaces.
-

5. User-Friendly Dashboard

- Centralized dashboard for uploading datasets, visualizing results, and managing prediction history.
 - Intuitive interface enables users without technical expertise to perform complex geospatial analyses.
-

6. Secure Access and Authentication

- Implements JWT-based authentication for secure login and user management.
 - Ensures that sensitive environmental data is protected from unauthorized access.
-

7. High Prediction Accuracy

- Machine Learning models are trained on historical environmental data to improve predictive accuracy.
 - Supports model evaluation using metrics such as accuracy, F1-score, RMSE, and confusion matrices.
-

8. Real-Time Decision Support

- Generates predictions and visualizations in near real-time, supporting timely environmental and disaster management decisions.
 - Useful for applications in agriculture, urban planning, climate risk analysis, and forest management.
-

9. Cost-Effective

- Uses open-source software such as Python, Flask, Scikit-learn, and Leaflet.js.
 - Reduces software licensing costs and development expenses.
-

10. Supports Sustainable Development Goals

- Enables monitoring of environmental resources and risk zones.
- Assists in making data-driven decisions for sustainable agriculture, forest conservation, disaster preparedness, and climate resilience.

6. FEASIBILITY STUDY

Feasibility study evaluates whether the proposed system can be successfully developed, deployed, and operated based on technical, economic, and operational considerations.

6.1 Technical Feasibility

The proposed system is technically feasible because it relies on well-supported, open-source technologies:

- **Python:** For data processing, Machine Learning, and backend development.
- **Flask:** Lightweight web framework for cloud deployment.
- **Scikit-learn and TensorFlow/Keras:** For implementing Machine Learning models.
- **JavaScript, Leaflet.js, and Chart.js:** For interactive visualization and mapping.

All required components are compatible, widely documented, and support scalable deployment.

6.2 Economic Feasibility

The system is economically feasible due to:

- Use of free and open-source software, eliminating licensing costs.
- Minimal hardware requirements; cloud platforms can be scaled based on user demand.
- Reduced development costs through the use of standard libraries and frameworks.

The cost-effective approach ensures that the system can be deployed and maintained within budget constraints.

6.3 Operational Feasibility

The system is operationally feasible because:

- It provides a **user-friendly web-based dashboard** for uploading datasets, running analyses, and viewing results.
- Users do not require advanced technical knowledge to operate the system.
- The system supports real-time environmental predictions and interactive visualizations, improving decision-making efficiency.
- Secure authentication ensures safe multi-user operation in cloud environments.

Overall, the proposed system is practical, scalable, and suitable for a variety of real-world environmental monitoring and geospatial analytics applications.

IMPLEMENTATION DETAILS

5 IMPLEMENTATION DETAILS

5.1 Overview of Implementation

Implementation is the phase where the system design is converted into a working model using programming

languages and tools. The proposed system is implemented using Python and follows a modular approach.

The system is divided into multiple components such as data preprocessing, machine learning model, backend API, and user interface. Each component is developed separately and then integrated to form a complete system.

5.2 Development Environment

- Programming Language: Python
- IDE: Visual Studio Code
- Libraries: Pandas, NumPy, Scikit-learn
- Framework: Flask
- Frontend: HTML, CSS, JavaScript

5.3 Dataset Implementation

The dataset used in this project consists of geospatial attributes such as:

- Latitude
- Longitude
- Rainfall
- Temperature
- Vegetation Index

The dataset is stored in CSV format and is processed using Python libraries.

Example Dataset Structure:

latitude, longitude, rainfall, temperature, vegetation, label

12.97,77.59,850,28,0.65,1

13.02,77.60,600,32,0.40,0

5.4 Data Preprocessing Implementation

Data preprocessing is a crucial step in machine learning.

Tasks performed:

- Handling missing values
- Selecting relevant features
- Formatting data

Code Logic Explanation:

```
import pandas as pd
```

```
def preprocess(df):
```

```
    df = df.fillna(0)
```

```
    return df[['latitude','longitude','rainfall','temperature','vegetation']]
```

Explanation:

- Missing values are replaced with 0
- Only required columns are selected
- Data is prepared for ML model

5.5 Machine Learning Model Implementation

Algorithm Used:

- Random Forest Classifier

Steps:

1. Load dataset
2. Split into training and testing data
3. Train model
4. Save trained model

Code Implementation:

```
import pandas as pd
```

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.ensemble import RandomForestClassifier
```

```
import joblib
```

```
data = pd.read_csv("../dataset/sample.csv")
```

```
X = data[['latitude','longitude','rainfall','temperature','vegetation']]
```

```
y = data['label']
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)
```

```
model = RandomForestClassifier()
```

```
model.fit(X_train, y_train)
```

```
joblib.dump(model, "geo_model.pkl")
```

Explanation:

- Data is split into training and testing sets

- Model is trained using Random Forest
- Model is saved for reuse

5.6 Backend Implementation (Flask API)

The backend is responsible for handling user requests and connecting the frontend with the ML model.

Code Implementation:

```
from flask import Flask, request, jsonify
import pandas as pd
import joblib
from preprocess import preprocess

app = Flask(__name__)
model = joblib.load("geo_model.pkl")

@app.route("/")
def home():
    return "Geospatial ML System Running"

@app.route("/predict", methods=["POST"])
def predict():
    file = request.files['file']
    df = pd.read_csv(file)

    processed = preprocess(df)
    predictions = model.predict(processed)

    return jsonify({"predictions": predictions.tolist()})

if __name__ == "__main__":
    app.run(debug=True)
```

Explanation:

- Flask creates API endpoints
- predict handles file upload
- Data is processed and passed to model
- Predictions returned as JSON

5.7 Frontend Implementation

The frontend provides a simple interface for user interaction.

HTML Code:

```
<!DOCTYPE html>
<html>
<head>
    <title>Geospatial ML System</title>
</head>
<body>
```

```
<h2>Upload Geospatial Data</h2>
<input type="file" id="file">
<button onclick="sendData()">Predict</button>
<pre id="output"></pre>

<script src="script.js"></script>
</body>
</html>
```

JavaScript Code:

```
function sendData() {
  let fileInput = document.getElementById("file");
  let formData = new FormData();
  formData.append("file", fileInput.files[0]);

  fetch("http://127.0.0.1:5000/predict", {
    method: "POST",
    body: formData
  })
  .then(res => res.json())
  .then(data => {
    document.getElementById("output").innerText =
      JSON.stringify(data, null, 2);
  });
}
```

Explanation:

- User uploads file
- JavaScript sends request to backend
- Output is displayed on screen

5.8 Integration of Modules

All modules are integrated as follows:

Frontend → Backend → Preprocessing → ML Model → Output

5.9 Working of the System

1. User opens application
2. Uploads dataset
3. Backend receives data
4. Data is preprocessed
5. ML model generates prediction
6. Output displayed

5.10 Key Features Implemented

- End-to-end ML pipeline

- File upload system
- Real-time prediction
- Modular architecture

5.11 Advantages of Implementation

- Simple and efficient
- Easy to extend
- Suitable for real-world applications
- Demonstrates ML + development skills

TESTING

6.1 Introduction to Testing

Testing is an important phase in software development used to verify that the system works correctly and meets the specified requirements. It ensures that the application is free from errors and performs efficiently under different conditions.

In this project, testing is performed at different levels to ensure the proper functioning of each module as well as the complete system.

6.2 Objectives of Testing

- To identify and fix errors
- To ensure system reliability
- To verify output accuracy
- To validate user inputs
- To ensure smooth integration of modules

6.3 Types of Testing

6.3.1 Unit Testing

Each module is tested individually.

Examples:

- Testing preprocessing function
- Testing ML model prediction

6.3.2 Integration Testing

Testing how different modules interact.

Example:

- Backend + ML model integration

6.3.3 System Testing

Testing the entire system workflow.

Example:

- Upload file → Get prediction

6.3.4 User Acceptance Testing (UAT)

Testing from user perspective.

Example:

- Checking if interface is easy to use

6.4 Test Cases

Test Case ID	Input	Expected Output	Result
TC01	Valid CSV file	Predictions generated	Pass
TC02	Missing values	Handled properly	Pass
TC03	Invalid format	Error message	Pass
TC04	Large dataset	Processed successfully	Pass

6.5 Error Handling

The system handles errors such as:

- Invalid file format
- Missing data
- Server errors

Proper error messages are displayed to guide the user.

6.6 Testing Results

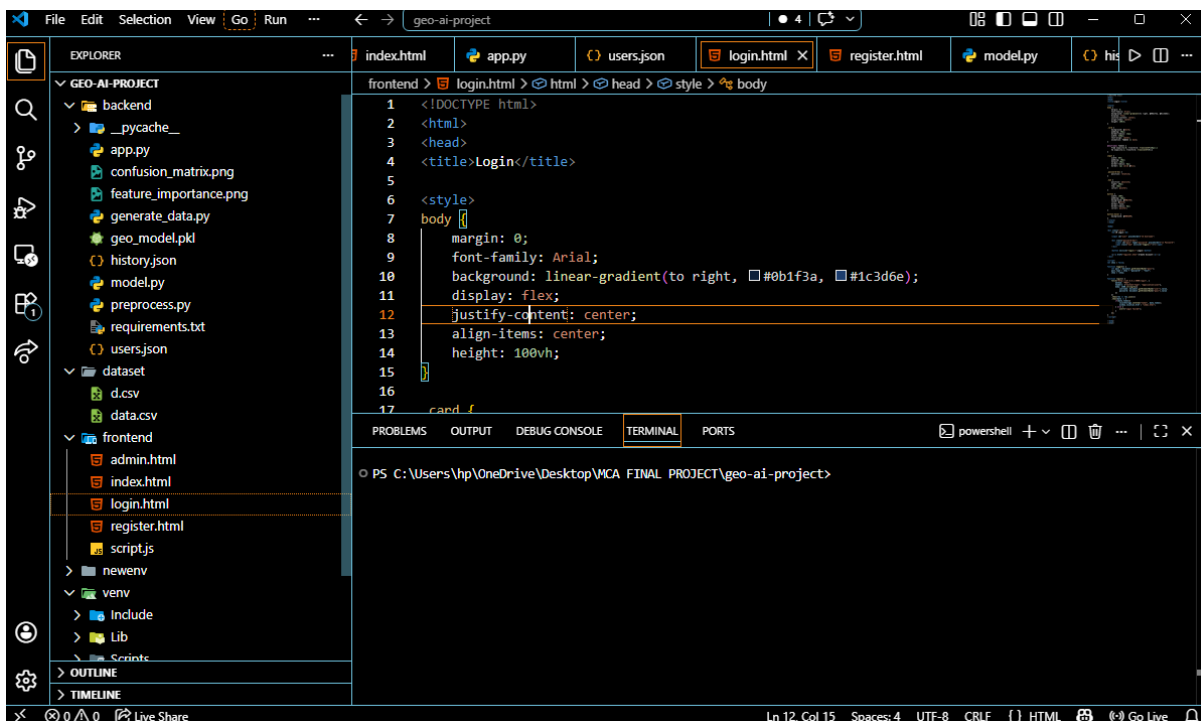
- All modules function correctly
- Predictions are generated successfully
- System performs efficiently

LIST OF REPORT

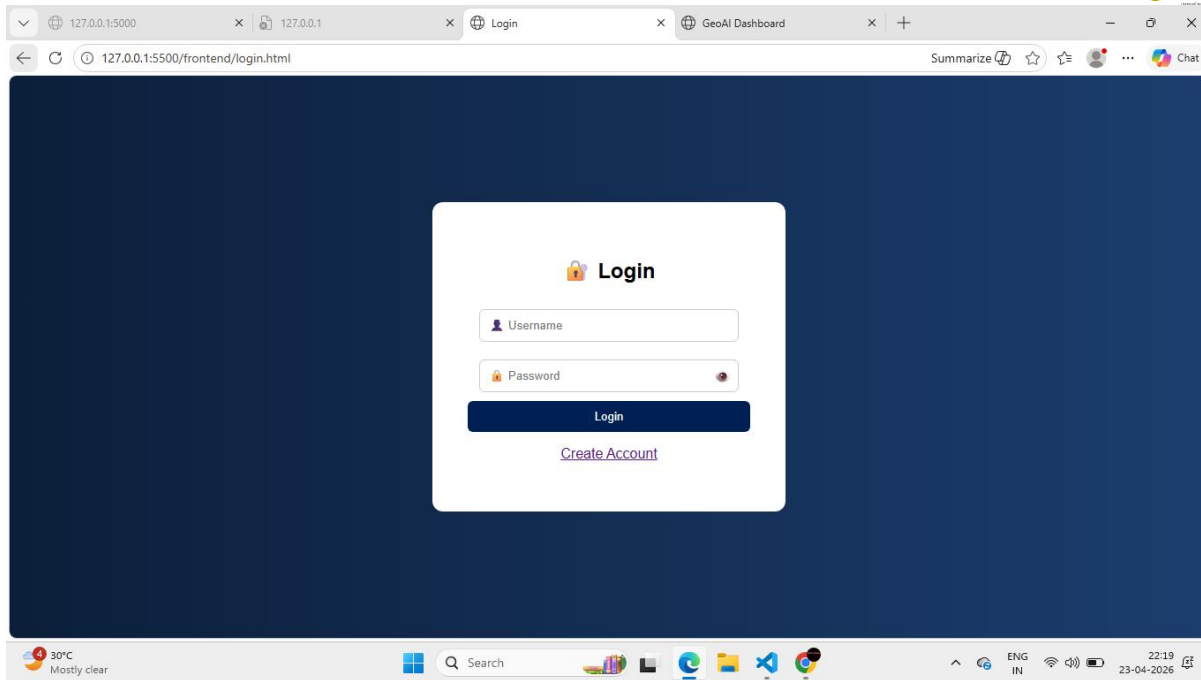
Input-Output

SCREENSHOTS

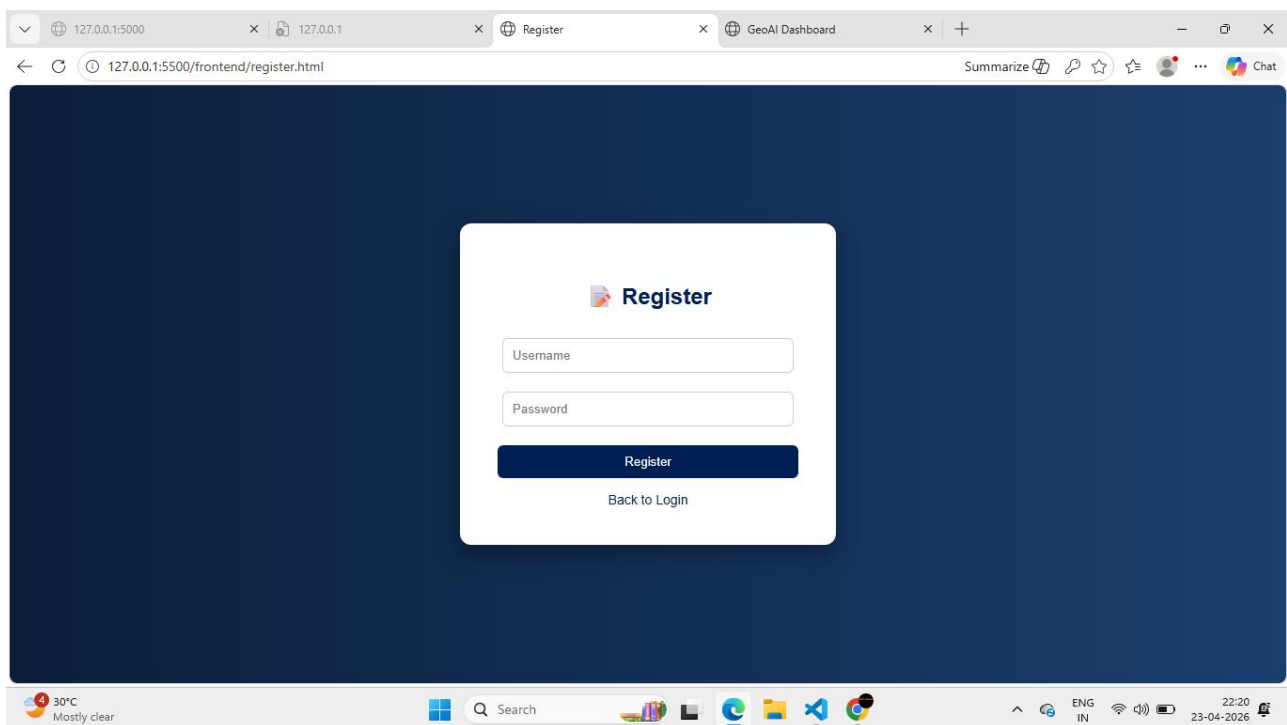
OUTPUT SCREENSHOTS:



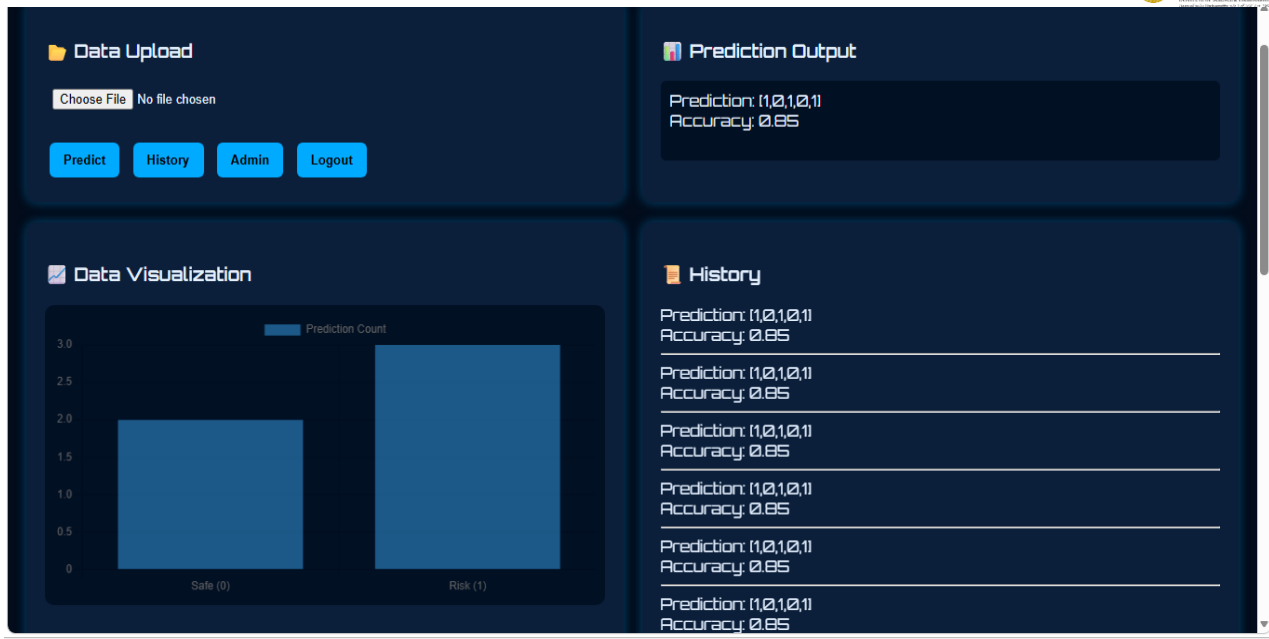
LOGIN PAGE



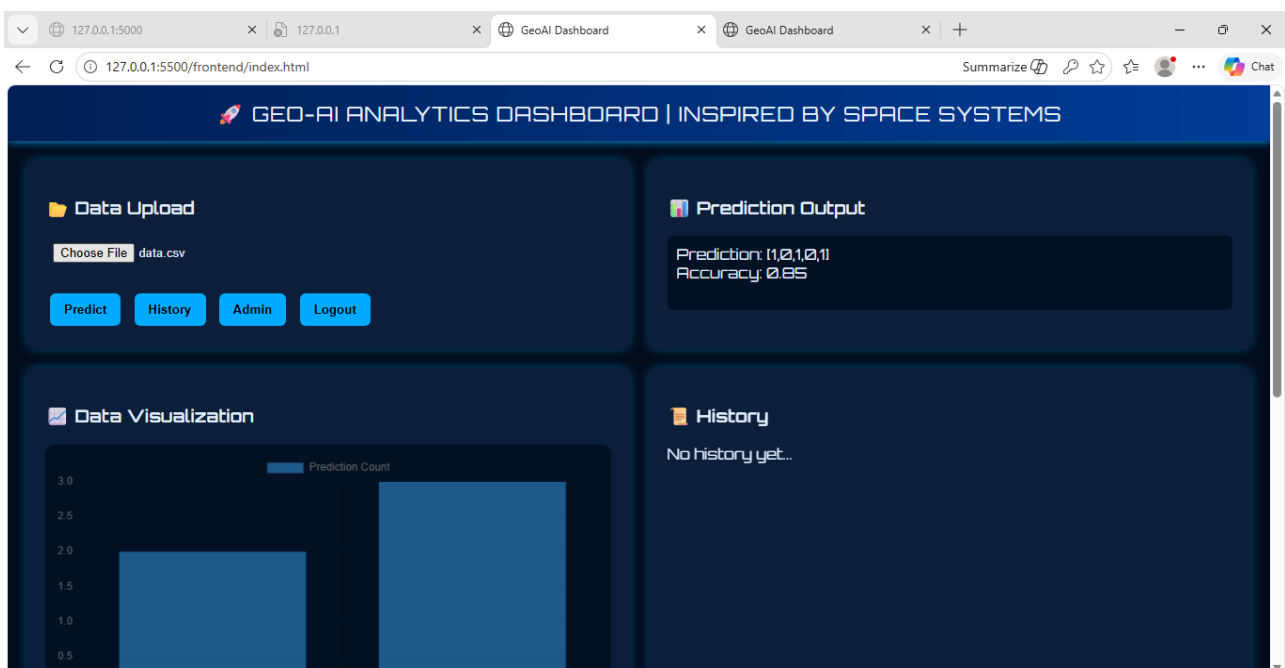
REGISTER PAGE



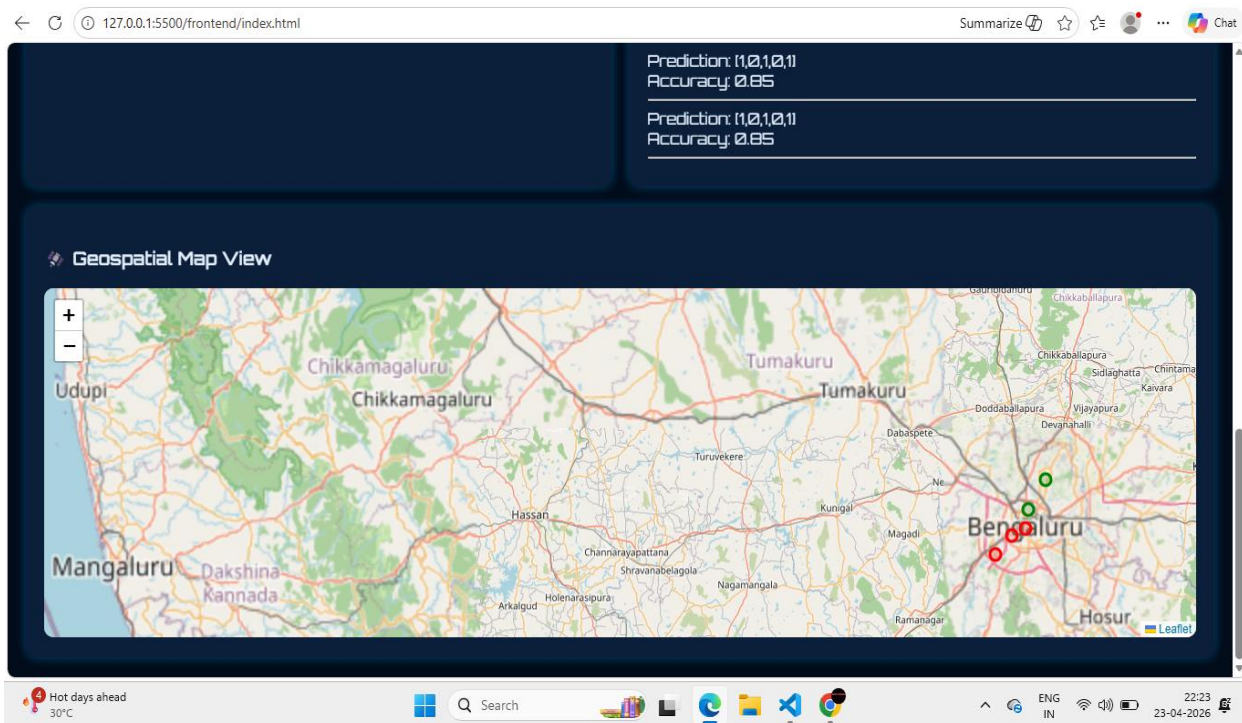
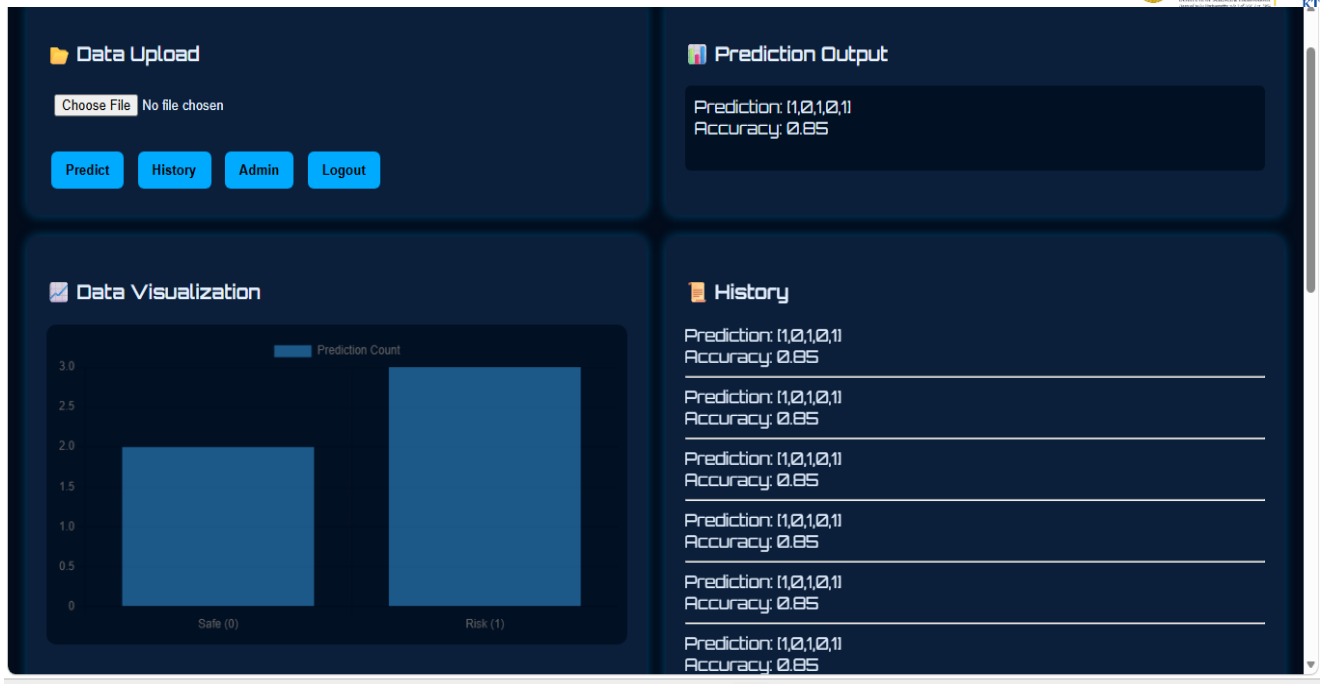
HOME PAGE

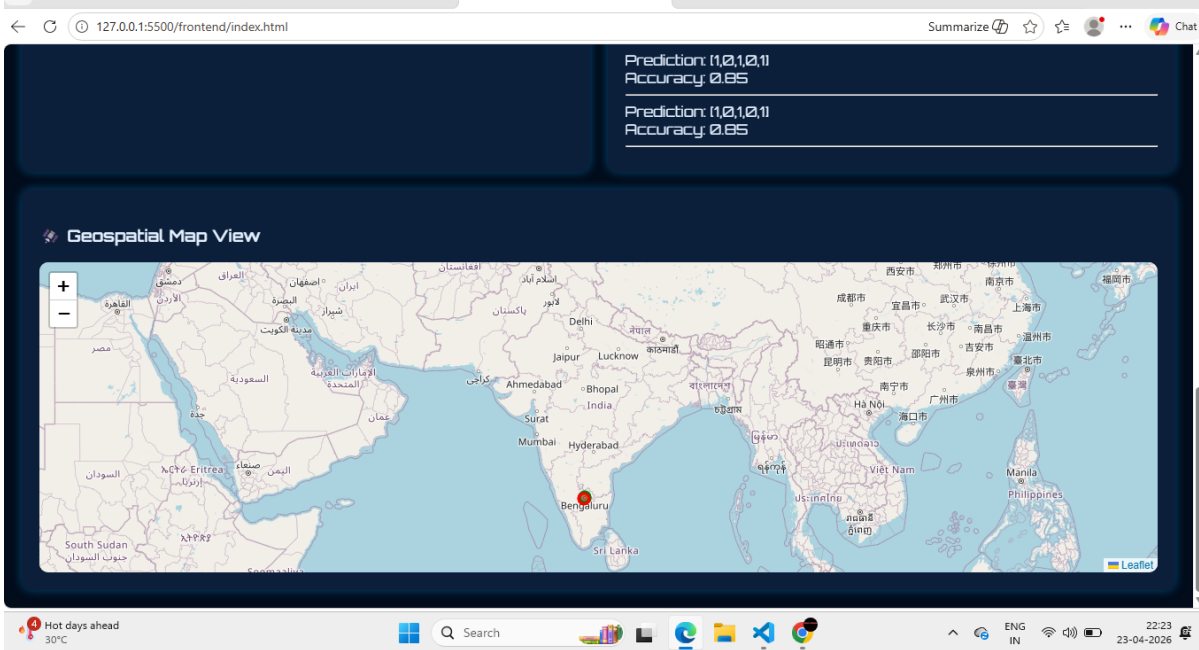


UPLOAD DATASET



DATA VISUALIZATION





CONCLUSION

6 CONCLUSION

The Geospatial Machine Learning System demonstrates the practical application of machine learning in analyzing environmental and spatial data for risk prediction. By utilizing geospatial attributes such as latitude, longitude, rainfall, temperature, and vegetation index, the system effectively identifies and classifies potential risk zones.

The integration of a web-based interface with a machine learning backend enables users to seamlessly upload datasets, generate predictions, and visualize results through interactive charts and maps. Features such as authentication, history tracking, and dashboard visualization enhance usability and make the system suitable for real-world applications.

The system architecture, based on a modular design and RESTful API framework, ensures scalability, flexibility, and efficient data flow between components. Visualization tools further improve interpretability, allowing users to derive meaningful insights from model outputs.

Although the current implementation is based on a limited dataset and baseline model, the project successfully establishes a strong foundation for geospatial intelligence systems. It highlights the potential of machine learning in supporting decision-making across domains such as disaster management, environmental monitoring, and urban planning.

In conclusion, this project validates the effectiveness of integrating machine learning with geospatial analytics and provides a scalable framework for developing advanced, data-driven environmental monitoring solutions.

Future Enhancements

To further enhance the system and evolve it into a production-grade platform, the following improvements are proposed:

Machine Learning Enhancements

- Implement advanced models such as Random Forest, XGBoost, and deep learning architectures
- Train on large-scale real-world geospatial datasets

- Apply feature engineering and hyperparameter tuning for improved accuracy

Geospatial & GIS Integration

- Integrate GIS platforms such as QGIS or ArcGIS
- Support raster and vector data processing
- Apply spatial clustering techniques (e.g., DBSCAN, K-Means) for hotspot detection

Real-Time Data Integration'

- Connect to live APIs for:
 - Weather data
 - Satellite imagery
 - Environmental and IoT sensor data

Explainable AI (XAI)

- Implement SHAP for feature importance analysis
- Provide model interpretability dashboards
- Enhance transparency for critical decision-making scenarios

Advanced Visualization & Dashboard

- Add filters (region, time, risk level)
- Enable comparative analytics
- Introduce heatmaps and temporal trend analysis
- Provide downloadable reports

Scalable Deployment

- Adopt microservices architecture
- Use containerization tools like Docker
- Deploy on cloud platforms such as AWS, Azure, or IBM Cloud

Security & Governance

- Implement role-based access control (RBAC)
- Apply strong password hashing and JWT security
- Ensure data encryption and compliance with privacy standards

REFERENCES

7 REFERENCES

1. Scikit-learn Documentation – <https://scikit-learn.org>
2. Flask Documentation – <https://flask.palletsprojects.com>
3. Pandas Documentation – <https://pandas.pydata.org>
4. Leaflet Documentation – <https://leafletjs.com>
5. Chart.js Documentation – <https://www.chartjs.org>
6. Kaggle. Datasets and machine learning resources. Retrieved from <https://www.kaggle.com>
7. HTML5. Web standards and documentation. Retrieved from <https://developer.mozilla.org>
8. JavaScript. Web development documentation. Retrieved from <https://developer.mozilla.org>
9. Geospatial Machine Learning literature including research journals and conference papers on spatial data analysis and predictive modelling
10. Indian Space Research Organisation. Remote Sensing and GIS Resources. Retrieved from <https://www.isro.gov.in>
11. Waldo Tobler (1970). *A Computer Movie Simulating Urban Growth in the Detroit Region*.
12. Michael F. Goodchild (2007). *Citizens as sensors: The world of volunteered geography*.
13. Indian Institute of Remote Sensing (IIRS), Indian Space Research Organisation. *Online Courses on Remote Sensing, GIS, and Geospatial Technologies*. Retrieved from <https://www.iirs.gov.in>

Appendices

Appendix A: Sample Dataset

latitude,longitude,rainfall,temperature,vegetation,label
12.97,75.59,850,28,0.55,0
13.02,78.60,400,32,0.40,0
12.95,77.55,920,27,0.70,0
13.10,77.65,400,35,0.30,1
12.90,77.50,880,26,0.68,1

The dataset used in this study consists of structured geospatial records with the following attributes:

Feature	Description
Latitude	Geographic coordinate (north-south position)
Longitude	Geographic coordinate (east-west position)
Rainfall	Annual rainfall (mm)
Temperature	Average temperature (°C)
Vegetation	Vegetation index (normalized value)
Label	Target variable (0 = Safe, 1 = Risk)

Appendix B: Technologies Used

- Frontend: HTML, CSS, JavaScript
 - Backend: Flask (Python)
 - Machine Learning: Scikit-learn
 - Data Handling: Pandas
 - Visualization:
 - Chart.js (graphs)
 - Leaflet.js (maps)
-

Appendix C: System Architecture

The system follows a **three-tier architecture**:

1. Presentation Layer

- Web interface using HTML, CSS, JavaScript
- User interaction and visualization

2. Application Layer

- Flask-based REST API
- Handles authentication, prediction requests, and data flow

3. Data & ML Layer

- Preprocessing module
- Trained ML model (Scikit-learn)
- Data storage (JSON-based persistence)

Appendix D: System Modules

1. User Authentication (Login/Register using JWT)
2. Data Upload Module (CSV input)
3. Prediction Engine (ML model)
4. Visualization Module (Charts + Maps)
5. History Tracking
6. Admin Panel

Appendix E: Algorithm Workflow

1. Input CSV dataset
2. Data preprocessing (cleaning, feature selection)
3. Feature transformation
4. Model inference
5. Output generation
6. Visualization (chart + map)

Appendix F: Project Workflow

1. Upload CSV file
2. Preprocess data
3. Apply ML model
4. Generate predictions
5. Display:
 - Output
 - Graph
 - Map
6. Save results in history

Appendix G: Evaluation Metrics

The system primarily uses:

- **Accuracy Score**
- Precision, Recall, F1-score, ROC-AUC

SOURCE CODE

Index.html

```
<!DOCTYPE html>
<html>
<head>
<title>GeoAI Dashboard</title>

<!-- Fonts -->
<link href="https://fonts.googleapis.com/css2?family=Orbitron:wght@400;600&display=swap"
rel="stylesheet">

<!-- Leaflet CSS -->
<link rel="stylesheet" href="https://unpkg.com/leaflet/dist/leaflet.css"/>

<style>

body {
  margin: 0;
  font-family: 'Orbitron', sans-serif;
  background: #020c1b;
  color: #e6f1ff;
}

/* HEADER */
.header {
  background: linear-gradient(90deg, #001f54, #003d99);
  padding: 15px;
  text-align: center;
  font-size: 22px;
  letter-spacing: 1px;
  box-shadow: 0 0 10px #00aaff;
}

/* GRID */
.dashboard {
  display: grid;
  grid-template-columns: 1fr 1fr;
  grid-gap: 20px;
  padding: 20px;
}

/* CARD */
.card {
```

```

    background: #0b1f3a;
    border-radius: 10px;
    padding: 20px;
    box-shadow: 0 0 12px rgba(0,170,255,0.4);
}

.full {
    grid-column: span 2;
}

/* BUTTON */
button {
    background: #00aaff;
    border: none;
    padding: 10px 15px;
    color: black;
    font-weight: bold;
    border-radius: 6px;
    cursor: pointer;
    margin: 5px;
}

button:hover {
    background: #33ccff;
}

/* INPUT */
input {
    padding: 8px;
    border-radius: 5px;
    border: none;
}

/* OUTPUT */
#output {
    margin-top: 10px;
    background: #021024;
    padding: 10px;
    border-radius: 6px;
    min-height: 60px; /* 🔄 prevents collapsing */
}

/* LOADER */
#loader {
    display: none;
    color: #00aaff;
}

/* MAP */
#map {
    height: 350px;
    border-radius: 10px;

```

```

}

/* CHART */
canvas {
  background: #021024;
  border-radius: 10px;
  padding: 10px;
}

</style>
</head>

<body>

<div class="header">
 GEO-AI ANALYTICS DASHBOARD | INSPIRED BY SPACE SYSTEMS
</div>

<div class="dashboard">

  <!-- INPUT -->
  <div class="card">
    <h3>&img alt="Folder icon" data-bbox="165 435 185 455"/> Data Upload</h3>

    <input type="file" id="file"><br><br>

    <!--  FIX: prevent form reload -->
    <button type="button" onclick="sendData(event)">Predict</button>
    <button type="button" onclick="loadHistory()">History</button>
    <button type="button" onclick="goAdmin()">Admin</button>
    <button type="button" onclick="logout()">Logout</button>

    <div id="loader">&img alt="Hourglass icon" data-bbox="265 600 285 620"/> Processing Data...</div>
  </div>

  <!-- OUTPUT -->
  <div class="card">
    <h3>&img alt="Bar chart icon" data-bbox="165 685 185 705"/> Prediction Output</h3>
    <div id="output">No results yet...</div>
  </div>

  <!-- CHART -->
  <div class="card">
    <h3>&img alt="Line chart icon" data-bbox="165 785 185 805"/> Data Visualization</h3>
    <canvas id="chart"></canvas>
  </div>

  <!-- HISTORY -->
  <div class="card">
    <h3>&img alt="Document icon" data-bbox="165 885 185 905"/> History</h3>
    <div id="history">No history yet...</div>
  </div>

```

```

</div>

<!-- MAP -->
<div class="card full">
  <h3>🗺️ Geospatial Map View</h3>
  <div id="map"></div>
</div>

</div>

<!-- LIBRARIES (ORDER MATTERS) -->
<script src="https://unpkg.com/leaflet/dist/leaflet.js"></script>
<script src="https://cdn.jsdelivr.net/npm/chart.js"></script>

<!-- YOUR JS -->
<script src="script.js"></script>

</body>
</html>

```

Login.html

```

<!DOCTYPE html>
<html>
<head>
<title>Login</title>

<style>
body {
  margin: 0;
  font-family: Arial;
  background: linear-gradient(to right, #0b1f3a, #1c3d6e);
  display: flex;
  justify-content: center;
  align-items: center;
  height: 100vh;
}

.card {
  background: white;
  padding: 40px;
  border-radius: 12px;
  width: 320px;
  text-align: center;
  animation: fadeIn 1s ease;
}

@keyframes fadeIn {
  from {opacity:0; transform: translateY(20px);}
  to {opacity:1; transform: translateY(0);}
}

```

```

input {

```

```

width: 85%;
padding: 10px;
margin: 10px;
border-radius: 6px;
border: 1px solid #ccc;
}

.password-box {
  position: relative;
}

.eye {
  position: absolute;
  right: 20px;
  top: 18px;
  cursor: pointer;
}

button {
  width: 100%;
  padding: 10px;
  background: #001f54;
  color: white;
  border: none;
  border-radius: 6px;
  cursor: pointer;
}

button:hover {
  background: #003d99;
}
</style>
</head>

<body>

<div class="card">
  <h2><img alt="lock icon" data-bbox="145 675 165 690"/> Login</h2>

  <input id="user" placeholder="<img alt="user icon" data-bbox="350 710 370 725"/> Username">

  <div class="password-box">
    <input id="pass" type="password" placeholder="<img alt="lock icon" data-bbox="515 760 535 775"/> Password">
    <span class="eye" onclick="toggle()"><img alt="eye icon" data-bbox="435 780 455 795"/></span>
  </div>

  <button onclick="login()">Login</button>

  <p><a href="register.html">Create Account</a></p>
</div>

```

```

<script>
let show = false;

function toggle() {
  let pass = document.getElementById("pass");
  pass.type = show ? "password" : "text";
  show = !show;
}

function login() {
  fetch("http://127.0.0.1:5000/login", {
    method: "POST",
    headers: {"Content-Type": "application/json"},
    body: JSON.stringify({
      username: document.getElementById("user").value,
      password: document.getElementById("pass").value
    })
  })
  .then(res => res.json())
  .then(data => {
    if(data.token){
      localStorage.setItem("token", data.token);
      window.location.href = "index.html";
    } else {
      alert("Login failed");
    }
  });
}
</script>

</body>
</html>

```

Register.html

```

<!DOCTYPE html>
<html>
<head>
<title>Register</title>

<style>
body {
  margin: 0;
  font-family: Arial;
  background: linear-gradient(to right, #0b1f3a, #1c3d6e);
  display: flex;
  justify-content: center;
  align-items: center;
  height: 100vh;
}

.card {

```

```
background: white;
padding: 40px;
border-radius: 12px;
width: 320px;
text-align: center;
box-shadow: 0 10px 25px rgba(0,0,0,0.3);
}
```

```
h2 {
margin-bottom: 20px;
color: #001f54;
}
```

```
input {
width: 90%;
padding: 10px;
margin: 10px 0;
border-radius: 6px;
border: 1px solid #ccc;
}
```

```
button {
width: 100%;
padding: 10px;
background: #001f54;
color: white;
border: none;
border-radius: 6px;
cursor: pointer;
margin-top: 10px;
}
```

```
button:hover {
background: #003d99;
}
```

```
a {
display: block;
margin-top: 15px;
color: #001f54;
text-decoration: none;
font-size: 14px;
}
```

```
</style>
</head>
```

```
<body>
```

```
<div class="card">
  <h2> Register</h2>
```

```

<input id="user" placeholder="Username">
<input id="pass" type="password" placeholder="Password">

<button onclick="register()">Register</button>

<a href="login.html">Back to Login</a>
</div>

```

```

<script>
function register() {
  fetch("http://127.0.0.1:5000/register", {
    method: "POST",
    headers: {"Content-Type": "application/json"},
    body: JSON.stringify({
      username: document.getElementById("user").value,
      password: document.getElementById("pass").value
    })
  })
  .then(res => res.json())
  .then(data => {
    alert(data.msg);
    window.location.href = "login.html";
  });
}
</script>

```

```

</body>
</html>

```

```

Admin.html
<!DOCTYPE html>
<html>
<head>
<title>Admin Dashboard</title>
<style>
body {
  margin: 0;
  font-family: Arial;
  background: linear-gradient(to right, #0b1f3a, #1c3d6e);

}

.card {
  background: white;
  padding: 40px;
  border-radius: 12px;
  width: 320px;
  text-align: center;
  box-shadow: 0 10px 25px rgba(0,0,0,0.3);
}

```

```

h2 {
  margin-bottom: 20px;
  color: #5baadf;
}

input {
  width: 90%;
  padding: 10px;
  margin: 10px 0;
  border-radius: 6px;
  border: 1px solid #ccc;
}

button {
  width: 100%;
  padding: 10px;
  background: #19db63;
  color: white;
  border: none;
  border-radius: 6px;
  cursor: pointer;
  margin-top: 10px;
}

button:hover {
  background: #030303;
}

a {
  display: block;
  margin-top: 15px;
  color: #001f54;
  text-decoration: none;
  font-size: 14px;
}

</style>
</head>

<body style="text-align:center; margin-top:50px;" >

<h2>Admin Dashboard</h2>

<button onclick="loadAdmin()">Load Data</button>

<div id="adminData"></div>

<script>
function loadAdmin() {
  fetch("http://127.0.0.1:5000/admin", {
    headers: {

```

```

        "Authorization": "Bearer " + localStorage.getItem("token")
    }
})
.then(res => res.json())
.then(data => {
    if(data.msg){
        alert(data.msg);
        return;
    }

    document.getElementById("adminData").innerHTML =
        "Total Users: " + data.total_users +
        "<br>Total Predictions: " + data.total_predictions +
        "<br>Users: " + data.users;
});
}
</script>

</body>
</html>

```

Login.html

```

<!DOCTYPE html>
<html>
<head>
<title>Login</title>

<style>
body {
    margin: 0;
    font-family: Arial;
    background: linear-gradient(to right, #0b1f3a, #1c3d6e);
    display: flex;
    justify-content: center;
    align-items: center;
    height: 100vh;
}

.card {
    background: white;
    padding: 40px;
    border-radius: 12px;
    width: 320px;
    text-align: center;
    animation: fadeIn 1s ease;
}

@keyframes fadeIn {
    from {opacity:0; transform: translateY(20px);}
    to {opacity:1; transform: translateY(0);}
}

```

```

input {
  width: 85%;
  padding: 10px;
  margin: 10px;
  border-radius: 6px;
  border: 1px solid #ccc;
}

.password-box {
  position: relative;
}

.eye {
  position: absolute;
  right: 20px;
  top: 18px;
  cursor: pointer;
}

button {
  width: 100%;
  padding: 10px;
  background: #001f54;
  color: white;
  border: none;
  border-radius: 6px;
  cursor: pointer;
}

button:hover {
  background: #003d99;
}
</style>
</head>

<body>

<div class="card">
  <h2><img alt="lock icon" data-bbox="148 708 168 723"/> Login</h2>

  <input id="user" placeholder="<img alt="user icon" data-bbox="353 743 373 758"/> Username">

  <div class="password-box">
    <input id="pass" type="password" placeholder="<img alt="lock icon" data-bbox="513 793 533 808"/> Password">
    <span class="eye" onclick="toggle()"><img alt="eye icon" data-bbox="433 813 453 828"/></span>
  </div>

  <button onclick="login()">Login</button>

  <p><a href="register.html">Create Account</a></p>

```

```

</div>

<script>
let show = false;

function toggle() {
  let pass = document.getElementById("pass");
  pass.type = show ? "password" : "text";
  show = !show;
}

function login() {
  fetch("http://127.0.0.1:5000/login", {
    method: "POST",
    headers: {"Content-Type": "application/json"},
    body: JSON.stringify({
      username: document.getElementById("user").value,
      password: document.getElementById("pass").value
    })
  })
  .then(res => res.json())
  .then(data => {
    if(data.token){
      localStorage.setItem("token", data.token);
      window.location.href = "index.html";
    } else {
      alert("Login failed");
    }
  });
}
</script>

</body>
</html>

```

Admin.html

```

<!DOCTYPE html>
<html>
<head>
<title>Admin Dashboard</title>
<style>
body {
  margin: 0;
  font-family: Arial;
  background: linear-gradient(to right, #0b1f3a, #1c3d6e);

}

```

```
.card {
  background: white;
  padding: 40px;
  border-radius: 12px;
  width: 320px;
  text-align: center;
  box-shadow: 0 10px 25px rgba(0,0,0,0.3);
}
```

```
h2 {
  margin-bottom: 20px;
  color: #5baadf;
}
```

```
input {
  width: 90%;
  padding: 10px;
  margin: 10px 0;
  border-radius: 6px;
  border: 1px solid #ccc;
}
```

```
button {
  width: 100%;
  padding: 10px;
  background: #19db63;
  color: white;
  border: none;
  border-radius: 6px;
  cursor: pointer;
  margin-top: 10px;
}
```

```
button:hover {
  background: #030303;
}
```

```
a {
  display: block;
  margin-top: 15px;
  color: #001f54;
  text-decoration: none;
  font-size: 14px;
}
```

```
</style>
</head>
```

```
<body style="text-align:center; margin-top:50px;" >
```

```
<h2>Admin Dashboard</h2>
```

```
<button onclick="loadAdmin()">Load Data</button>
```

```
<div id="adminData"></div>
```

```
<script>
```

```
function loadAdmin() {  
  fetch("http://127.0.0.1:5000/admin", {  
    headers: {  
      "Authorization": "Bearer " + localStorage.getItem("token")  
    }  
  })  
  .then(res => res.json())  
  .then(data => {  
    if(data.msg){  
      alert(data.msg);  
      return;  
    }  
  })
```

```
  document.getElementById("adminData").innerHTML =  
    "Total Users: " + data.total_users +  
    "<br>Total Predictions: " + data.total_predictions +  
    "<br>Users: " + data.users;
```

```
});
```

```
}
```

```
</script>
```

```
</body>
```

```
</html>
```

App.py

```
from flask import Flask, request, jsonify  
from flask_cors import CORS  
from flask_jwt_extended import (  
    JWTManager, create_access_token,  
    jwt_required, get_jwt_identity  
)
```

```
import json, os
```

```
import pandas as pd
```

```
import joblib
```

```
from preprocess import preprocess
```

```
app = Flask(__name__)
```

```
@app.route("/")
```

```
def home():
```

```
    return "Backend is running 🚀"
```

```
app.config["JWT_SECRET_KEY"] = "this_is_a_super_secure_secret_key_1234567890_abcdef"
```

```
jwt = JWTManager(app)
```

```
CORS(app)
```

```
model = joblib.load("geo_model.pkl")
```

```

# ----- USERS -----
def load_users():
    if not os.path.exists("users.json"):
        return {}
    with open("users.json", "r") as f:
        return json.load(f)

def save_users(users):
    with open("users.json", "w") as f:
        json.dump(users, f)

# ----- HISTORY -----
def load_history():
    if not os.path.exists("history.json"):
        return []
    with open("history.json", "r") as f:
        return json.load(f)

def save_history(data):
    with open("history.json", "w") as f:
        json.dump(data, f)

# ----- REGISTER -----
@app.route("/register", methods=["POST"])
def register():
    data = request.json
    users = load_users()

    if data["username"] in users:
        return jsonify({"msg": "User exists"}), 400

    users[data["username"]] = data["password"]
    save_users(users)

    return jsonify({"msg": "Registered successfully"})

# ----- LOGIN -----
@app.route("/login", methods=["POST"])
def login():
    data = request.json
    users = load_users()

    if data["username"] in users and users[data["username"]] == data["password"]:
        token = create_access_token(identity=data["username"])
        return jsonify({"token": token})

    return jsonify({"msg": "Invalid credentials"}), 401

# ----- PREDICT -----
@app.route("/predict", methods=["POST"])
@jwt_required()

```

```

def predict():
    print("Prediction API HIT")

    user = get_jwt_identity()

    if 'file' not in request.files:
        return jsonify({"error": "No file uploaded"}), 400

    file = request.files['file']

    if file.filename == "":
        return jsonify({"error": "Empty file"}), 400

    df = pd.read_csv(file)

    processed = preprocess(df)
    predictions = model.predict(processed)

    history = load_history()
    history.append({
        "user": user,
        "predictions": predictions.tolist()
    })
    save_history(history)

    return jsonify({
        "prediction": predictions.tolist(),
        "data": df.to_dict('records'),
        "accuracy": 0.85
    })

# ----- HISTORY -----
@app.route("/history", methods=["GET"])
@jwt_required()
def get_history():
    user = get_jwt_identity()
    history = load_history()

    user_history = [h for h in history if h["user"] == user]
    return jsonify(user_history)

# ----- ADMIN -----
@app.route("/admin", methods=["GET"])
@jwt_required()
def admin():
    user = get_jwt_identity()

    if user != "admin":
        return jsonify({"msg": "Access denied"}), 403

    users = load_users()
    history = load_history()

```

```

return jsonify({
    "total_users": len(users),
    "total_predictions": len(history),
    "users": list(users.keys())
})

if __name__ == "__main__":
    app.run(debug=True)

```

generate.py

```

import pandas as pd
import random

data = []

for i in range(100):
    lat = round(random.uniform(12.5, 13.5), 2)
    lon = round(random.uniform(77.0, 78.0), 2)
    rainfall = random.randint(400, 1000)
    temp = random.randint(25, 40)
    veg = round(random.uniform(0.2, 0.8), 2)

    # Simple logic for label
    label = 1 if veg > 0.5 else 0

    data.append([lat, lon, rainfall, temp, veg, label])

df = pd.DataFrame(data, columns=[
    "latitude", "longitude", "rainfall", "temperature", "vegetation", "label"
])

df.to_csv("../dataset/data1.csv", index=False)

print("Dataset created!")

```

model.py

```

import pandas as pd
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, confusion_matrix
import joblib
import matplotlib.pyplot as plt

data = pd.read_csv("../dataset/data.csv")

X = data[['latitude', 'longitude', 'rainfall', 'temperature', 'vegetation']]

```

```

y = data['label']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

model = RandomForestClassifier()
model.fit(X_train, y_train)

y_pred = model.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)

cm = confusion_matrix(y_test, y_pred)
print("Confusion Matrix:\n", cm)

# Plot confusion matrix
plt.imshow(cm)
plt.title("Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")

for i in range(len(cm)):
    for j in range(len(cm)):
        plt.text(j, i, cm[i][j])

plt.savefig("confusion_matrix.png")
# plt.show()

# Feature importance
plt.figure()
plt.bar(X.columns, model.feature_importances_)
plt.title("Feature Importance")
plt.savefig("feature_importance.png")
# plt.show()

preprocess.py

import pandas as pd

def preprocess(df):
    df = df.fillna(0)
    return df[['latitude', 'longitude', 'rainfall', 'temperature', 'vegetation']]

script.js
// ===== DEBUG =====
console.log("JS Loaded");

// ===== SEND DATA =====
function sendData(event) {

```

```

if (event) event.preventDefault(); // 🚫 prevent refresh

let fileInput = document.getElementById("file");

if (fileInput.files.length === 0) {
  alert("Upload CSV first!");
  return;
}

let token = localStorage.getItem("token");
if (!token) {
  alert("Please login first!");
  window.location.href = "login.html";
  return;
}

document.getElementById("loader").style.display = "block";

let formData = new FormData();
formData.append("file", fileInput.files[0]);

fetch("http://127.0.0.1:5000/predict", {
  method: "POST",
  headers: {
    "Authorization": "Bearer " + token
  },
  body: formData
})
.then(res => res.json())
.then(data => {
  document.getElementById("loader").style.display = "none";

  if (data.error || data.msg) {
    document.getElementById("output").innerText =
      data.error || data.msg;
    return;
  }

  let prediction = data.prediction || [];
  let accuracy = data.accuracy || "N/A";
  let dataRows = data.data || [];

  let outputText =
    "Prediction: " + JSON.stringify(prediction) +
    "\nAccuracy: " + accuracy;

  // ✅ Persist output
  localStorage.setItem("lastOutput", outputText);
  localStorage.setItem("lastPredictions", JSON.stringify(prediction));
  localStorage.setItem("lastData", JSON.stringify(dataRows));
  localStorage.setItem("lastAccuracy", accuracy);
  window.lastOutput = outputText;

```

```

document.getElementById("output").innerText = outputText;

// ✅ Save clean history (NO undefined)
if (prediction.length > 0) {
  saveHistory(prediction, accuracy);
  drawChart(prediction);
  drawMap(prediction, dataRows);
}
})
.catch(err => {
  console.error(err);
  document.getElementById("loader").style.display = "none";
  document.getElementById("output").innerText = "Server error!";
});
}

// ===== CHART =====
function drawChart(predictions) {
  let canvas = document.getElementById("chart");

  // 🔄 FIX: ensure canvas visible
  canvas.style.display = "block";

  if (window.chartInstance) {
    window.chartInstance.destroy();
  }

  let count1 = predictions.filter(x => x === 1).length;
  let count0 = predictions.filter(x => x === 0).length;

  window.chartInstance = new Chart(canvas, {
    type: "bar",
    data: {
      labels: ["Safe (0)", "Risk (1)"],
      datasets: [{
        label: "Prediction Count",
        data: [count0, count1]
      }]
    },
    options: {
      responsive: true
    }
  });
}

// ===== MAP =====
function drawMap(predictions, data) {
  let mapDiv = document.getElementById("map");

  // 🔄 clear old map
  mapDiv.innerHTML = "";

```

```

window.map = L.map('map').setView([20.5937, 78.9629], 4);

L.tileLayer('https://{s}.tile.openstreetmap.org/{z}/{x}/{y}.png')
.addTo(window.map);

predictions.forEach((p, i) => {
  let lat = data[i].latitude;
  let lon = data[i].longitude;

  let color = p === 1 ? "red" : "green";

  L.circleMarker([lat, lon], {
    color: color,
    radius: 6
  })
  .addTo(window.map)
  .bindPopup("Prediction: " + p + "<br>Lat: " + lat + "<br>Lon: " + lon);
});

// 🐛 FIX: force map render
setTimeout(() => {
  window.map.invalidateSize();
}, 200);
}

// ===== HISTORY =====
function loadHistory() {
  let historyDiv = document.getElementById("history");

  let history = JSON.parse(localStorage.getItem("history")) || [];

  // 🐛 FIX: remove bad entries
  history = history.filter(h => h.prediction && h.accuracy);

  if (history.length === 0) {
    historyDiv.innerHTML = "No history found";
    return;
  }

  historyDiv.innerHTML = history.map(h =>
    `<div>Prediction: ${JSON.stringify(h.prediction)} <br> Accuracy: ${h.accuracy}</div><hr>`
  ).join("");
}

// ===== SAVE HISTORY =====
function saveHistory(prediction, accuracy) {
  let history = JSON.parse(localStorage.getItem("history")) || [];

  // 🐛 avoid saving undefined
  if (!prediction || !accuracy) return;

  history.push({ prediction, accuracy });
}

```

```

    localStorage.setItem("history", JSON.stringify(history));
}

// ===== ADMIN =====
function goAdmin() {
    window.location.href = "admin.html";
}

// ===== LOGOUT =====
function logout() {
    localStorage.removeItem("token");
    window.location.href = "login.html";
}

// ===== ON LOAD =====
window.onload = function() {
    let lastOutput = localStorage.getItem("lastOutput");
    if (lastOutput) {
        document.getElementById("output").innerText = lastOutput;
    }

    let lastPred = localStorage.getItem("lastPredictions");
    let lastData = localStorage.getItem("lastData");
    if (lastPred && lastData) {
        let pred = JSON.parse(lastPred);
        let data = JSON.parse(lastData);
        drawChart(pred);
        drawMap(pred, data);
    }
};

```

CANDIDATE DECLARATION

Project Title: **GEOSPATIAL DATA ANALYSIS USING MACHINE LEARNING**

I, **SAKSHI SRIVASTAVA**

(Enrollment number: **EA2432251010614**), declare that this project is my original work developed as part of my academic study and research with guidance and learning support from the Indian Institute of Remote Sensing (IIRS-ISRO) course materials and related educational resources. I have properly acknowledged all references, learning materials, software tools, research papers, websites, reports, lecture notes, and other resources used during the preparation of this project.

I further certify that this project has not been previously submitted for assessment in any academic institution and that I have not copied, fully or partially, the work of any other individual without proper acknowledgment. Any assistance received through online courses, tutorials, documentation, or educational guidance has been used only for learning and reference purposes during the development of this project.

I confirm that I have identified and declared all possible conflicts that I may have.

Signed and Submitted:

Date: 09 MAY,2026

